

2.3 Properties of Functions

Functions that satisfy one or both of two special properties can be very useful in solving a variety of computational problems. One property is that distinct elements map to distinct elements. The other property is that the range is equal to the codomain. We'll discuss these properties in more detail and give some examples where they are useful.

2.3.1 Injections and Surjections

Injective Property

A function $f: A \rightarrow B$ is called *injective* (also *one-to-one*, or an *embedding*) if it maps distinct elements of A to distinct elements of B . Another way to say this is that f is injective if $x \neq y$ implies $f(x) \neq f(y)$. Yet another way to say this is that f is injective if $f(x) = f(y)$ implies $x = y$. An injective function is called an *injection*.

For example, [Figure 2.9](#) illustrates an injection from a set A to a set B .

Surjective Property

A function $f: A \rightarrow B$ is called *surjective* (also *onto*) if the range of f is the codomain B . Another way to say this is that f is surjective if each element $b \in B$ can be written as $b = f(x)$ for some element $x \in A$. A surjective function is called a *surjection*.

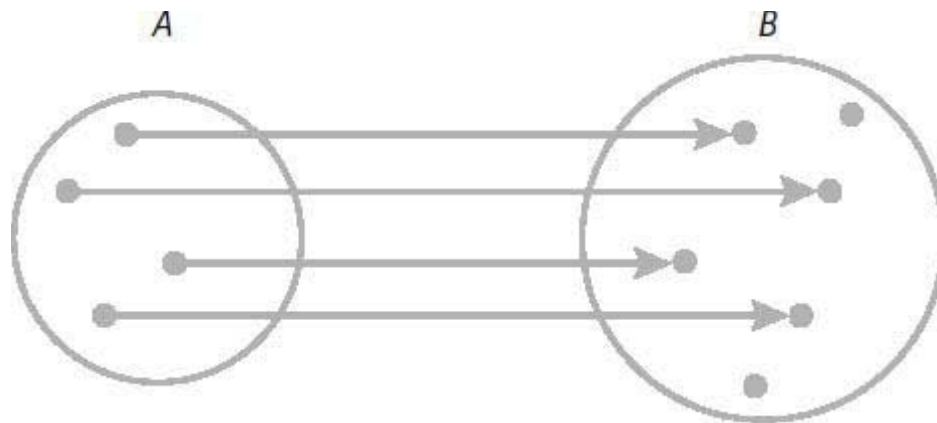


Figure 2.9 An injection.

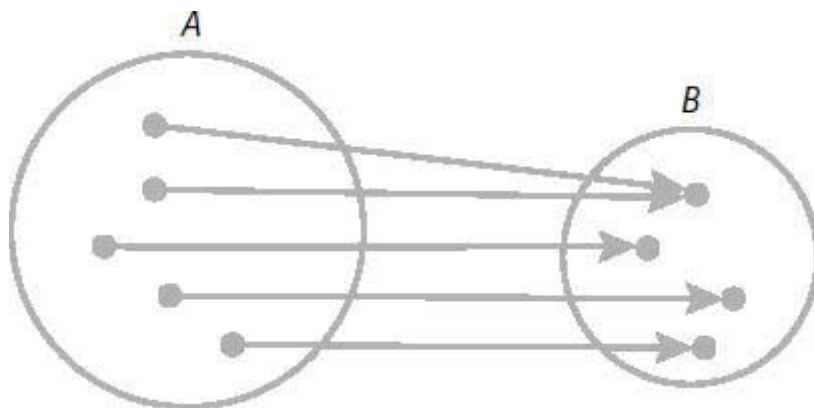


Figure 2.10 A surjection.

For example, [Figure 2.10](#) pictures a surjection from A to B .

example 2.17 Injective or Surjective

We'll give a few examples of functions that have one or the other of the injective and surjective properties.

1. The function $f: \mathbb{R} \rightarrow \mathbb{Z}$ defined by $f(x) = \lfloor x + 1 \rfloor$ is surjective because for any $y \in \mathbb{Z}$ there is a number in $\mathbb{R}$, namely $y - 1$, such that $f(y - 1) = y$. But f is not injective because, for example, $f(3.5) = f(3.6)$.
2. The function $f: \mathbb{N}_8 \rightarrow \mathbb{N}_8$ defined by $f(x) = 2x \bmod 8$ is not injective because, for example, $f(0) = f(4)$. f is not surjective because the range of f is only the set $\{0, 2, 4, 6\}$.
3. Let $g: \mathbb{N} \rightarrow \mathbb{N} \times \mathbb{N}$ be defined by $g(x) = (x, x)$. Then g is injective because if $x, y \in \mathbb{N}$ and $x \neq y$, then $g(x) = (x, x) \neq (y, y) = g(y)$. But g is not surjective because, for example, nothing maps to $(0, 1)$.
4. The function $f: \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ defined by $f(x, y) = 2x + y$ is surjective. To see this, notice that any $z \in \mathbb{N}$ is either even or odd. If z is even, then $z = 2k$ for some $k \in \mathbb{N}$, so $f(k, 0) = z$. If z is odd, then $z = 2k + 1$ for some $k \in \mathbb{N}$, so $f(k, 1) = z$. Thus f is surjective. But f is not injective because, for example, $f(0, 2) = f(1, 0)$.

end example

2.3.2 Bijections and Inverses

A function is called *bijective* if it is both injective and surjective. Another term for bijective is “one-to-one and onto.” A bijective function is called a *bijection* or a “one-to-one correspondence.”

For example, [Figure 2.11](#) pictures a bijection from A to B .

example 2.18 A Bijection

Let $(0, 1) = \{x \in \mathbb{R} \mid 0 < x < 1\}$ and let $\mathbb{R}^+$ denote the set of positive real numbers. We'll show that the function $f: (0, 1) \rightarrow \mathbb{R}^+$ defined by

$$f(x) = \frac{x}{1-x}$$

is a bijection. To show that f is an injection, let $f(x) = f(y)$. Then

$$\frac{x}{1-x} = \frac{y}{1-y},$$

which can be cross multiplied to get $x - xy = y - xy$. Subtract $-xy$ from both sides to get $x = y$. Thus f is injective. To show that f is surjective, let $y > 0$ and try to find $x \in (0, 1)$ such that $f(x) = y$. We'll solve the equation

$$\frac{x}{1-x} = y.$$

Cross multiply and solve for x to obtain

$$x = \frac{y}{y+1}.$$

It follows that $f(y/(y+1)) = y$, and since $y > 0$, it follows that $0 < y/(y+1) < 1$. Thus f is surjective. Therefore, f is a bijection.

end example

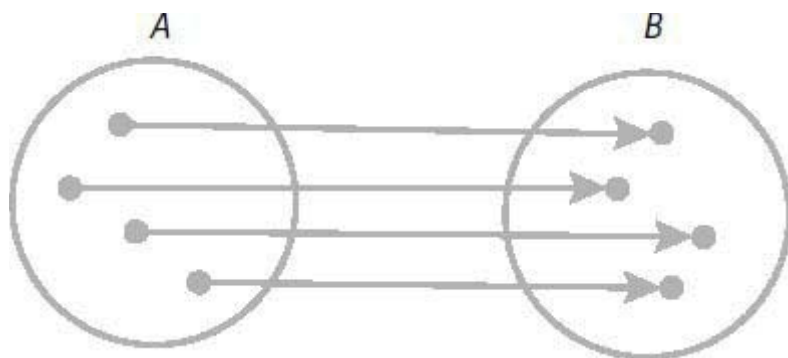


Figure 2.11 A bijection.

Inverse Functions

Bijections always come in pairs. If $f: A \rightarrow B$ is a bijection, then there is a function $g: B \rightarrow A$, called the *inverse* of f , defined by $g(b) = a$ if $f(a) = b$. Of course, the inverse of f is also a bijection and we have $g(f(a)) = a$ for all $a \in A$ and $f(g(b)) = b$ for all $b \in B$. In other words, $g \circ f = \text{id}_A$ and $f \circ g = \text{id}_B$.

We should note that there is exactly one inverse of any bijection f . Suppose that g and h are two inverses of f . Then for any $x \in B$ we have

$$\begin{aligned}
 g(x) &= g(\text{id}_B(x)) \\
 &= g(f(h(x))) && (\text{since } f \circ h = \text{id}_B) \\
 &= \text{id}_A(h(x)) && (\text{since } g \circ f = \text{id}_A) \\
 &= h(x).
 \end{aligned}$$

This tells us that $g = h$. The inverse of f is often denoted by the symbol f^{-1} . So if f is a bijection and $f(a) = b$, then $f^{-1}(b) = a$. Notice the close relationship between the equation $f^{-1}(b) = a$ and the pre-image equation $f^{-1}(\{b\}) = \{a\}$.

example 2.19 Inverses

1. For any real number $b > 1$ the log function $\log_b x$ and the exponential function b^x are inverses of each other because $\log_b b^x = x$ and $b^{\log_b x} = x$.
2. Let *Odd* and *Even* be the sets of odd and even natural numbers,

respectively. The function $f: \text{Odd} \rightarrow \text{Even}$ defined by $f(x) = x - 1$ is a bijection. Check it out. The inverse of f can be defined by $f^{-1}(x) = x + 1$. Notice that $f^{-1}(f(x)) = f^{-1}(x - 1) = (x - 1) + 1 = x$.

3. The function $f: \mathbb{N}_5 \rightarrow \mathbb{N}_5$ defined by $f(x) = 2x \bmod 5$ is bijective because, $f(0) = 0, f(1) = 2, f(2) = 4, f(3) = 1$, and $f(4) = 3$. The inverse of f can be defined by $f^{-1}(x) = 3x \bmod 5$. For example, $f^{-1}(f(4)) = 3f(4) \bmod 5 = 9 \bmod 5 = 4$. Check the other values too.

end example

The fact that the function $f: \mathbb{N}_5 \rightarrow \mathbb{N}_5$ defined by $f(x) = 2x \bmod 5$ (in the preceding example) is a bijection is an instance of an interesting and useful fact about the mod function and inverses. Here is the result.

The Mod Function and Inverses

(2.6)

Let $n \geq 1$ and let $f: \mathbb{N}_n \rightarrow \mathbb{N}_n$ be defined as follows, where a and b are integers.

$$f(x) = (ax + b) \bmod n.$$

Then f is a bijection if and only if $\gcd(a, n) = 1$. When this is the case, the inverse function f^{-1} is defined by

$$f^{-1}(x) = (kx + c) \bmod n,$$

where c is an integer such that $f(c) = 0$, and k is an integer such that $1 = ak + nm$ for some integer m .

Proof: We'll prove the iff part of the statement and leave the form of the inverse as an exercise. If $n = 1$, then $f: \{0\} \rightarrow \{0\}$ is a bijection and $\gcd(a, 1) = 1$. So we'll let $n > 1$.

Assume that f is a bijection and show that $\gcd(a, n) = 1$. Then f is surjective, so there are numbers $s, c \in \mathbb{N}_n$ such that $f(s) = 1$ and $f(c) = 0$. Using

the definition of f , these equations become

$$(as + b) \bmod n = 1 \text{ and } (ac + b) \bmod n = 0.$$

Therefore, there are integers q_1 and q_2 such that the two equations become

$$as + b + nq_1 = 1 \text{ and } ac + b + nq_2 = 0.$$

Solve the second equation for b to get $b = -ac - nq_2$, and substitute for b in the first equation to get

$$1 = a(s - c) + n(q_1 - q_2).$$

Since $\gcd(a, n)$ divides both a and n , it divides the right side of the equation and so must also divide 1. Therefore, $\gcd(a, n) = 1$.

Now assume that $\gcd(a, n) = 1$ and show that f is a bijection. Since $\mathbb{N}_n$ is finite, we need only show that f is an injection to conclude that it is a bijection. So let $x, y \in \mathbb{N}_n$ and let $f(x) = f(y)$. Then

$$(ax + b) \bmod n = (ay + b) \bmod n,$$

which by (2.4a) implies that n divides $(ax + b) - (ay + b)$. Therefore, n divides $a(x - y)$, and since $\gcd(a, n) = 1$, we conclude from (2.2d) that n divides $x - y$. But the only way for n to divide $x - y$ is for $x - y = 0$ because both $x, y \in \mathbb{N}_n$. Thus $x = y$, and it follows that f is injective, hence also surjective, and therefore bijective. QED.

Relationships

An interesting property that relates injections and surjections is that if there is an injection from A to B , then there is a surjection from B to A , and conversely. A less surprising fact is that if the composition $f \circ g$ makes sense and if both f and g have one of the properties injective, surjective, or bijective, then $f \circ g$ also has the property. We'll list these facts for the record.

- a. If f and g are injective, then $g \circ f$ is injective.
- b. If f and g are surjective, then $g \circ f$ is surjective.
- c. If f and g are bijective, then $g \circ f$ is bijective.
- d. There is an injection from A to B if and only if there is a surjection from B to A .

Proof: We'll prove (2.7d) and leave the others as exercises. Suppose that f is an injection from A to B . We'll define a function g from B to A . Since f is an injection, it follows that for each $b \in \text{range}(f)$ there is exactly one $a \in A$ such that $b = f(a)$. In this case, we define $g(b) = a$. For each $b \in B - \text{range}(f)$ we have the freedom to let g map b to any element of A that we like. So g is a function from B to A and we defined g so that $\text{range}(g) = A$. Thus g is surjective.

For the other direction, assume that g is a surjection from B to A . We'll define a function f from A to B . Since g is a surjection, it follows that for each $a \in A$, the pre-image $g^{-1}(\{a\}) \neq \emptyset$. So we can pick an element $b \in g^{-1}(\{a\})$ and define $f(a) = b$. Thus f is a function from A to B . Now if $x, y \in A$ and $x \neq y$, then $g^{-1}(\{x\}) \cap g^{-1}(\{y\}) = \emptyset$. Since $f(x) \in g^{-1}(\{x\})$ and $f(y) \in g^{-1}(\{y\})$, it follows that $f(x) \neq f(y)$. Thus f is injective. QED.

2.3.3 The Pigeonhole Principle

We're going to describe a useful rule that we often use without thinking. For example, suppose 21 pieces of mail are placed in 20 mailboxes. Then one mailbox receives at least two pieces of mail. This is an example of the *pigeonhole principle*, where we think of the pieces of mail as pigeons and the mailboxes as pigeonholes.

Pigeonhole Principle

If m pigeons fly into n pigeonholes where $m > n$, then one pigeonhole will have two or more pigeons.

We can describe the pigeonhole principle in more formal terms as follows: If A and B are finite sets with $|A| > |B|$, then every function from A to B maps at least

two elements of A to a single element of B . This is the same as saying that no function from A to B is an injection.

This simple idea is used often in many different settings. We'll be using it at several places in the book.

example 2.20 Pigeonhole Examples

Here are a few sample statements that can be justified by the pigeonhole principle.

1. The “musical chairs” game is played with n people and $n - 1$ chairs for them to sit on when the music stops.
2. In a group of eight people, two were born on the same day of the week.
3. If a six-sided die is tossed seven times, one side will come up twice.
4. If a directed graph with n vertices has a path of length n or longer, then the path must pass through some vertex at least twice. This implies that the graph contains a cycle.
5. In any set of $n + 1$ integers, there are two numbers that have the same remainder on division by n . This follows because there are only n remainders possible on division by n .
6. The decimal expansion of any rational number contains a repeating sequence of digits (they might be all zeros). For example, $359/495 = 0.7252525 \dots$, $7/3 = 2.333 \dots$, and $2/5 = 0.4000 \dots$. To see this, let m/n be a rational number. Divide m by n until all the digits of m are used up. This gets us to the decimal point. Now continue the division by n for $n + 1$ more steps. This gives us $n + 1$ remainders. Since there are only n remainders possible on division by n , the pigeonhole principle tells us that one of the remainders will be repeated. So the sequence of remainders between the repeated remainders will be repeated forever. This causes the corresponding sequence of digits in the decimal expansion to be repeated forever.

end example

2.3.4 Simple Ciphers

Bijections and inverse functions play an important role when working with systems (called ciphers) to encipher and decipher information. We'll give a few examples to illustrate the connections. For ease of discussion we'll denote the 26 letters of the lowercase alphabet by the set $\mathbb{N}_{26} = \{0, 1, 2, \dots, 25\}$, where we identify a with 0, b with 1, and so on.

To get things started we'll describe a cipher that enciphers a message by adding 5 to each letter. For example, the message 'abc' becomes 'fgh'. The cipher is easy to write once we figure out how to wrap around the end of the alphabet. For example, to shift the letter z (i.e., 25) by 5 letters we need to come up with the letter e (i.e., 4). All we need to do is add the two numbers mod 26:

$$(25 + 5) \bmod 26 = 4.$$

So we can define the cipher as a function f as follows:

$$f(x) = (x + 5) \bmod 26.$$

It follows from (2.6) that f is a bijection of type $\mathbb{N}_{26} \rightarrow \mathbb{N}_{26}$. So we have a cipher for transforming letters. For example, the message 'hello' transforms to 'mjqqt'. To decipher the message we need to reverse the process, which is easy in this case because the inverse of f is easy to guess.

$$f^{-1}(x) = (x - 5) \bmod 26.$$

For example, to see that e maps back to z , we can observe that 4 maps to 25.

$$f^{-1}(4) = (4 - 5) \bmod 26 = (-1) \bmod 26 = 25.$$

A cipher *key* is the information needed to encipher or decipher a message. The cipher we've been talking about is called an *additive* cipher and its key is 5. A cryptanalyst who intercepts the message 'mjqqt' can easily check whether it was created by an additive cipher. An additive cipher is an example of a *monoalphabetic* cipher, which is a cipher that replaces repeated occurrences of a character in a message by the same character from the cipher alphabet.

A *multiplicative* cipher is a monoalphabetic cipher that enciphers each

letter by using a multiplier as a key. For example, suppose we define the cipher

$$g(x) = 3x \bmod 26.$$

In this case the key is 3. For example, this cipher maps a to a , c to g , and m to k . Is g a bijection? You can convince yourself that it is by exhaustive checking. But it's easier to use (2.6). Since $\gcd(3, 26) = 1$ it follows that g is a bijection. What about deciphering? Again, (2.6) comes to the rescue to tell us the form of g^{-1} . Since we can write $\gcd(3, 26) = 1 = 3(9) + 26(-1)$, and since $g(0) = 0$, it follows that we can define g^{-1} as

$$g^{-1}(x) = 9x \bmod 26.$$

There are some questions to ask about multiplicative ciphers. Which keys act as an identity (not changing the message)? Is there always one letter that never changes no matter what the key? Do fractions work as keys? What about decoding (i.e., deciphering) a message? Do you need a new deciphering algorithm?

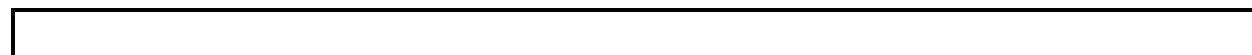
An *affine* cipher is a monoalphabetic cipher that combines additive and multiplicative ciphers. For example, with the pair of keys (3, 5) we can define f as

$$f(x) = (3x + 5) \bmod 26.$$

We can use (2.6) to conclude that f is a bijection because $\gcd(3, 26) = 1$. So we can also decipher messages with f^{-1} , which we can construct using (2.6) as

$$f^{-1}(x) = (9x + 7) \bmod 26.$$

Some ciphers leave one or more letters fixed. For example, an additive cipher that shifts by a multiple of 26 will leave all letters fixed. A multiplicative cipher always sends 0 to 0, so one letter is fixed. But what about an affine cipher of the form $f(x) = (ax + b) \bmod 26$? When can we be sure that no letters are fixed? In other words, when can we be sure that $f(x) \neq x$ for all $x \in \mathbb{N}_{26}$? The answer is, when $\gcd(a - 1, 26)$ does not divide b . Here is the general result that we've been discussing.



The Mod Function and Fixed Points

(2.8)

Let $n \geq 1$ and let $f: \mathbb{N}_n \rightarrow \mathbb{N}_n$ be defined as follows, where a and b are integers.

$$f(x) = (ax + b) \bmod n.$$

Then f has no fixed points (i.e., f changes every letter of an alphabet) if and only if $\gcd(a - 1, n)$ does not divide b .

This result follows from an old and easy result from number theory, and we'll discuss it in the exercises. Let's see how the result helps our cipher problem.

example 2.21 Simple Ciphers

The function $f(x) = (3x + 5) \bmod 26$ does not have any fixed points because $\gcd(3 - 1, 26) = \gcd(2, 26) = 2$, and 2 does not divide 5. It's nice to know that we don't have to check all 26 values of f .

On the other hand, the function $f(x) = (3x + 4) \bmod 26$ has fixed points because $\gcd(3 - 1, 26) = 2$ and 2 divides 4. For this example, we can observe that $f(11) = 11$ and $f(24) = 24$. So in terms of our association of letters with numbers we would have $f(l) = l$ and $f(y) = y$.

end example

Whatever cipher we use, we always have some questions: Is it a bijection? What is the range of values for the keys? Is it hard to decipher an intercepted message?

2.3.5 Hash Functions

Suppose we wish to retrieve some information stored in a table of size n with indexes $0, 1, \dots, n - 1$. The items in the table can be very general things. For example, the items might be strings of letters, or they might be large records with many fields of information. To look up a table item we need a *key* to the information we desire.

For example, if the table contains records of information for the 12 months of the year, the keys might be the three-letter abbreviations for the 12 months. To look up the record for January, we would present the key Jan to a lookup program. The program uses the key to find the table entry for the January record of information. Then the information would be available to us.

An easy way to look up the January record is to search the table until the key Jan is found. This might be OK for a small table with 12 entries. But it may be impossibly slow for large tables with thousands of entries. Here is the general problem that we want to solve:

Given a key, find the table entry containing the key without searching.

This may seem impossible at first glance. But let's consider a way to use a function to map each key directly to its table location.

Definition of a Hash Function

A *hash function* is a function that maps a set S of keys to a finite set of table indexes, which we'll assume are $0, 1, \dots, n - 1$. A table whose information is found by a hash function is called a *hash table*.

example 2.22 A Hash Function

Let S be the set of three-letter abbreviations for the months of the year. We might define a hash function $f: S \rightarrow \{0, 1, \dots, 11\}$ in the following way.

$$f(XYZ) = (\text{ord}(X) + \text{ord}(Y) + \text{ord}(Z)) \bmod 12,$$

where $\text{ord}(X)$ denotes the integer value of the ASCII code for X . (The ASCII values for A to Z and a to z are 65 to 90 and 97 to 122, respectively.) For example, we'll compute the value for the key Jan.

$$\begin{aligned} f(\text{Jan}) &= (\text{ord}(J) + \text{ord}(a) + \text{ord}(n)) \bmod 12 \\ &= (74 + 97 + 110) \bmod 12 \\ &= 5. \end{aligned}$$

Most programming languages have efficient implementations of the `ord` and `mod` functions, so hash functions constructed from them are quite fast. Here is the listing of all the values of f .

Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
5	5	0	3	7	1	11	9	8	6	7	4

end example

We should notice the function f in Example 2.22 is not injective. For example, $f(\text{Jan}) = f(\text{Feb}) = 5$. So if we use f to construct a hash table, we can't put the information for January and February at the same address. Let's discuss this problem.

Collisions

If a hash function is injective, then it maps every key to the index of the hash table where the information is stored and no searching is involved. Often this is not possible. When two keys map to the same table index, the result is called a *collision*. So if a hash function is not injective, it has collisions. In Example 2.22 the hash function has collisions $f(\text{Jan}) = f(\text{Feb})$ and $f(\text{May}) = f(\text{Nov})$.

When collisions occur, we store the information for one of the keys in the common table location and must find some other location for the other keys. There are many ways to find the location for a key that has collided with another key. One technique is called *linear probing*. With this technique the program searches the remaining locations in a “linear” manner. For example, if location k is the collision index, then the following sequence of table locations is searched.

$$(k + 1) \bmod n, (k + 2) \bmod n, (k + 3) \bmod n, \dots$$

example 2.23 A Hash Table

We'll use the hash function f from Example 2.22 to construct a hash table for the months of the year by placing the three-letter abbreviations in the table one by one, starting with Jan and continuing to Dec. We'll use linear probing to

resolve collisions that occur in the process. For example, since $f(\text{Jan}) = 5$, we place Jan in position 5 of the table. Next, since $f(\text{Feb}) = 5$ and since position 5 is full, we look for the next available position and place Feb in position 6. Continuing in this way, we eventually construct the following hash table, where entries in parentheses need some searching to be found.

0	1	2	3	4	5	6	7	8	9	10	11
Mar	Jun	(Nov)	Apr	Dec	Jan	(Feb)	May	Sep	Aug	(Oct)	Jul

There are many questions. Can we find an injection so there are no collisions? If we increased the size of the table, would it give us a better chance of finding an injection? If the table size is increased, can we scatter the elements so that collisions can be searched for in less time?

end example

Searching successive locations may not be the best way to resolve collisions if it takes too many comparisons to find an open location. A simple alternative is to use a “gap” between table locations in order to “scatter” or “hash” the information to other parts of the table. Let g be a gap, where $1 \leq g < n$. Then the following sequence of table locations is searched in case a collision occurs at location k :

$$(k + g) \bmod n, (k + 2g) \bmod n, (k + 3g) \bmod n, \dots$$

Some problems can occur if we’re not careful with our choice of g . For example, suppose $n = 12$ and $g = 4$. Then the search sequence can skip some table entries. For example, if $k = 7$, then the first 12 indexes in the previous sequence are

$$11, 3, 7, 11, 3, 7, 11, 3, 7, 11, 3, 7.$$

So we would miss table entries 0, 1, 2, 4, 5, 6, 8, 9, and 10. Let’s try another value for g . Suppose we try $g = 5$. If $k = 7$, then the first 12 indexes in the previous sequence are

$$0, 5, 10, 3, 8, 1, 6, 11, 4, 9, 2, 7.$$

In this case we cover the entire set $\{0, 1, \dots, 11\}$.

Can we always find a search sequence that hits all the elements of $\{0, 1, \dots, n-1\}$? Happily, the answer is yes. Just pick g and n so that $\gcd(g, n) = 1$. Then (2.6) tells us that the function f defined by $f(x) = (gx + k) \bmod n$ is a bijection. For example, if we pick n to be a prime number, then $\gcd(g, n) = 1$ for any g in the interval $1 \leq g < n$. That's why table sizes are often prime numbers, even though the data set may have fewer entries than the table size.

There are many ways to define hash functions and to resolve collisions. The paper by Cichelli [1980] examines some bijective hash functions.

Exercises

Injections, Surjections, and Bijections

1. The `fatherOf` function from *People* to *People* is neither injective nor surjective. Why?
2. For each of the following cases, construct a function satisfying the given condition, where the domain and codomain are chosen from the sets

$$A = \{a, b, c\}, B = \{x, y, z\}, C = \{1, 2\}.$$

- a. Injective but not surjective.
 - b. Surjective but not injective.
 - c. Bijective.
 - d. Neither injective nor surjective.
3. For each of the following types, compile some statistics: the number of functions of that type; the number that are injective; the number that are surjective; the number that are bijective; the number that are neither injective, surjective, nor bijective.
 - a. $\{a, b, c\} \rightarrow \{1, 2\}$.
 - b. $\{a, b\} \rightarrow \{1, 2, 3\}$.
 - c. $\{a, b, c\} \rightarrow \{1, 2, 3\}$.
 4. Show that each function $f: \mathbb{N} \rightarrow \mathbb{N}$ has the listed properties.

- a. $f(x) = 2x$. (injective and not surjective)
- b. $f(x) = x + 1$. (injective and not surjective)
- c. $f(x) = \text{floor}(x/2)$. (surjective and not injective)
- d. $f(x) = \text{ceiling}(\log_2(x + 1))$. (surjective and not injective)
- e. $f(x) = \text{if } x \text{ is odd then } x - 1 \text{ else } x + 1$. (bijective)

5. Determine whether each function is injective or surjective.

- a. $f: \mathbb{R} \rightarrow \mathbb{Z}$, where $f(x) = \text{floor}(x)$.
- b. $f: \mathbb{N} \rightarrow \mathbb{N}$, where $f(x) = x \bmod 10$.
- c. $f: \mathbb{Z} \rightarrow \mathbb{N}$ defined by $f(x) = |x + 1|$.
- d. $\text{seq}: \mathbb{N} \rightarrow \text{lists}(\mathbb{N})$.
- e. $\text{dist}: A \times \text{lists}(B) \rightarrow \text{lists}(A \times B)$.
- f. $f: A \rightarrow \text{power}(A)$, A is any set, and $f(x) = \{x\}$.
- g. $f: \text{lists}(A) \rightarrow \text{power}(A)$, A is finite, and $f(\langle x_1, \dots, x_n \rangle) = \{x_1, \dots, x_n\}$.
- h. $f: \text{lists}(A) \rightarrow \text{bags}(A)$, A is finite, and $f(x_1, \dots, x_n) = [x_1, \dots, x_n]$.
- i. $f: \text{bags}(A) \rightarrow \text{power}(A)$, A is finite, and $f(\langle x_1, \dots, x_n \rangle) = \{x_1, \dots, x_n\}$.

6. Let $\mathbb{R}^+$ and $\mathbb{R}^-$ denote the sets of positive and negative real numbers, respectively. If $a, b \in \mathbb{R}$ and $a < b$, let $(a, b) = \{x \in \mathbb{R} \mid a < x < b\}$. Show that each of the following functions is a bijection.

- a. $f: (0, 1) \rightarrow (a, b)$ defined by $f(x) = (b - a)x + a$.
- b. $f: \mathbb{R}^+ \rightarrow (0, 1)$ defined by $f(x) = 1/(x + 1)$.
- c. $f: (1/2, 1) \rightarrow \mathbb{R}^+$ defined by $f(x) = 1/(2x - 1) - 1$.
- d. $f: (0, 1/2) \rightarrow \mathbb{R}^-$ defined by $f(x) = 1/(2x - 1) + 1$.
- e. $f: (0, 1) \rightarrow \mathbb{R}$ defined by

$$f(x) = \begin{cases} 1/(2x - 1) - 1 & \text{if } 1/2 < x < 1 \\ 0 & \text{if } x = 1/2 \\ 1/(2x - 1) + 1 & \text{if } 0 < x < 1/2 \end{cases}.$$

The Pigeonhole Principle

7. Use the pigeonhole principle for each of the following statements.
- a. How many people are needed in a group to say that three were born on the same day of the week?
 - b. How many people are needed in a group to say that four were born in the same month?
 - c. Why does any set of 10 nonempty strings over $\{a, b, c\}$ contain two different strings whose starting letters agree and whose ending letters agree?
 - d. Find the size needed for a set of nonempty strings over $\{a, b, c, d\}$ to contain two strings whose starting letters agree and whose ending letters agree.
8. Use the pigeonhole principle to verify each of the following statements.
- a. In any set of 11 natural numbers, there are two numbers whose decimal representations contain a common digit.
 - b. In any set of four numbers picked from the set $\{1, 2, 3, 4, 5, 6\}$, there are two numbers whose sum is seven.
 - c. If five distinct numbers are chosen from the set $\{1, 2, 3, 4, 5, 6, 7, 8\}$, then two of the numbers chosen are consecutive (i.e., of the form n and $n + 1$). *Hint:* List the five numbers chosen as, x_1, x_2, x_3, x_4, x_5 and list the successors as $x_1 + 1, x_2 + 1, x_3 + 1, x_4 + 1, x_5 + 1$. Are there more than eight numbers listed?

Simple Ciphers and the Mod Function

9. Each of the following functions has the form $f(x) = (ax + b) \bmod n$. Assume that each function has type $\mathbb{N}_n \rightarrow \mathbb{N}_n$, so that we can think of f as a cipher for an alphabet represented by the numbers $0, 1, \dots, n - 1$. Use (2.6) to determine whether each function is a bijection, and, if so, construct its inverse. Then use (2.8) to determine whether the function has fixed points (i.e., letters that don't change), and, if so, find them.
- a. $f(x) = 2x \bmod 6$.
 - b. $f(x) = 2x \bmod 5$.

- c. $f(x) = 5x \bmod 6$.
 - d. $f(x) = (3x + 2) \bmod 6$.
 - e. $f(x) = (2x + 3) \bmod 7$.
 - f. $f(x) = (5x + 3) \bmod 12$.
 - g. $f(x) = (25x + 7) \bmod 16$.
10. Think of the letters A to Z as the numbers 0 to 25 and let f be a cipher of the form $f(x) = (ax + b) \bmod 26$.
- a. Use (2.6) to find all values of a ($0 \leq a < 26$) that will make f bijective.
 - b. For the values of a in Part (a) that make f bijective, use (2.8) to find a general statement about the values of b ($0 \leq b < 26$) that will ensure that f maps each letter to a different letter.

Hash Functions

11. Let $S = \{\text{one, two, three, four, five, six, seven, eight, nine}\}$ and let $f: S \rightarrow \mathbb{N}_9$ be defined by $f(x) = (3|x|) \bmod 9$, where $|x|$ means the number of letters in x . For each of the following gaps, construct a hash table that contains the strings of S by choosing a string for entry in the table by the order that it is listed in S . Resolve collisions by linear probing with the given gap and observe whether all strings can be placed in the table.
- a. Gap = 1.
 - b. Gap = 2.
 - c. Gap = 3.
12. Repeat Exercise 11 for the set $S = \{\text{Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday}\}$ and the function $f: S \rightarrow \mathbb{N}_7$ defined by $f(x) = (2|x| + 3) \bmod 7$.
13. Repeat Exercise 11 for the set $S = \{\text{January, February, March, April, May, June, July, August}\}$ and $f: S \rightarrow \mathbb{N}_8$ defined by $f(x) = (|x| + 3) \bmod 8$.

Proofs and Challenges

14. Find integers a and b such that the function $f: \mathbb{N}_{12} \rightarrow \mathbb{N}_{12}$ defined by $f(x) = (ax + b) \bmod 12$ is bijective and $f^{-1} = f$.

15. Let $f: A \rightarrow B$ and $g: B \rightarrow C$. Prove each of the following statements.
- If f and g are injective, then $g \circ f$ is injective.
 - If f and g are surjective, then $g \circ f$ is surjective.
 - If f and g are bijective, then $g \circ f$ is bijective.
16. Let f and g be bijections of type $A \rightarrow A$ such that $g(f(x)) = x$ for all $x \in A$. Prove that $f(g(x)) = x$ for all $x \in A$.
17. Assume that the functions f and g can be formed into a composition $g \circ f$.
- If $g \circ f$ is surjective, what can you say about f or g ?
 - If $g \circ f$ is injective, what can you say about f or g ?
18. Let $g: A \rightarrow B$ and $h: A \rightarrow C$ and let f be defined by $f(x) = (g(x), h(x))$. Show that each of the following statements holds.
- If f is surjective, then g and h are surjective. Find an example to show that the converse is false.
 - If g or h is injective, then f is injective. Find an example to show that the converse is false.
19. Prove that the equation $ax \bmod n = b \bmod n$ has a solution x if and only if $\gcd(a, n)$ divides b .
20. Use the result of Exercise 19 to prove (2.8): Let $n \geq 1$ and $f: \mathbb{N}_n \rightarrow \mathbb{N}_n$ be defined by $f(x) = (ax + b) \bmod n$. Then f has no fixed points if and only if $\gcd(a - 1, n)$ does not divide b .
21. Prove the second part of (2.6). In other words, assume the following facts.
- $f: \mathbb{N}_n \rightarrow \mathbb{N}_n$ is defined by $f(x) = (ax + b) \bmod n$.
 f is a bijection, which also means that $\gcd(a, n) = 1$.
 c is an integer such that $f(c) = 0$.
 k is an integer such that $1 = ak + nm$ for some integer m .
 $g: \mathbb{N}_n \rightarrow \mathbb{N}_n$ is defined by $g(x) = (kx + c) \bmod n$.
- Prove that $g = f^{-1}$.

2.4 Countability

Let's have a short discussion about counting sets that may not be finite. We'll have to examine what it means to count an infinite set and what it means to compare the size of infinite sets. In so doing we'll find some useful techniques that can be applied to questions in computer science. For example, we'll see as a consequence of our discussions that there are some limits on what can be computed. We'll start with some simplifying notation.

2.4.1 Comparing the Size of Sets

Let A and B be sets. If there is a bijection between A and B , we'll denote the fact by writing

$$|A| = |B|.$$

In this case we'll say that A and B have the *same size* or have the *same cardinality*, or are *equipotent*.

example 2.24 Cardinality of a Finite Set

Let $A = \{(x+1)^3 \mid x \in \mathbb{N} \text{ and } 1 \leq (x+1)^3 \leq 3000\}$. Let's find the cardinality of A . After a few calculations we can observe that

$$(0+1)^3 = 1, (1+1)^3 = 8, \dots, (13+1)^3 = 2744, \text{ and } (14+1)^3 = 3375.$$

So we have a bijection $f: \{0, 1, \dots, 13\} \rightarrow A$, where $f(x) = (x+1)^3$. Therefore, $|A| = |\{0, 1, \dots, 13\}| = 14$.

end example

example 2.25 Cardinality of an Infinite Set

Let Odd denote the set of odd natural numbers. Then the function $f: \mathbb{N} \rightarrow Odd$ defined by $f(x) = 2x + 1$ is a bijection. So Odd and $\mathbb{N}$ have the same size and we write $|Odd| = |\mathbb{N}|$.

If there is an injection from A to B , we'll denote the fact by writing

$$|A| \leq |B|.$$

In this case we'll say that the size, or cardinality, of A is *less than or the same as* that of B . Recall that there is an injection from A to B if and only if there is a surjection from B to A . So $|A| \leq |B|$ also means that there is a surjection from B to A .

If there is an injection from A to B but *no* bijection between them, we'll denote the fact by writing

$$|A| < |B|.$$

In this case we'll say that the size, or cardinality, of A is *less than* that of B . So $|A| < |B|$ means that $|A| \leq |B|$ and $|A| \neq |B|$.

2.4.2 Sets That Are Countable

Informally, a set is countable if its elements can be counted in a step-by-step fashion (e.g., count one element each second), even if it takes as many seconds as there are natural numbers. Let's clarify the idea by relating sets that can be counted to subsets of the natural numbers.

If A is a finite set with n elements, then we can represent the elements of A by listing them in the following sequence:

$$x_0, x_1, x_2, \dots, x_{n-1}.$$

If we associate each x_k with the subscript k , we get a bijection between A and the set $\{0, 1, \dots, n-1\}$.

Suppose A is an infinite set such that we can represent all the elements of A by listing them in the following infinite sequence:

$$x_0, x_1, x_2, \dots, x_n, \dots$$

If we associate each x_k with the subscript k , we get a bijection between A and the set $\mathbb{N}$ of natural numbers.

Definition of Countable

The preceding descriptions give us the seeds for a definition of countable. A set is *countable* if it is finite or if there is a bijection between it and $\mathbb{N}$. In the latter case, the set is said to be *countably infinite*. In terms of size we can say that a set S is countable if $|S| = |\{0, 1, \dots, n-1\}|$ for some natural number n or $|S| = |\mathbb{N}|$. If a set is not countable, it is said to be *uncountable*.

Countable Properties

(2.9)

- a. Every subset of $\mathbb{N}$ is countable.
- b. S is countable if and only if $|S| \leq |\mathbb{N}|$.
- c. Any subset of a countable set is countable.
- d. Any image of a countable set is countable.

Proof. We'll prove (a) and (b) and leave (c) and (d) as exercises. Let S be a subset of $\mathbb{N}$. If S is finite, then it is countable by definition. So assume that S is infinite. Now since S is a set of natural numbers, it has a smallest element that we'll represent by x_0 . Next, we'll let x_1 be the smallest element of the set $S - \{x_0\}$. We'll continue in this manner, letting x_n be the smallest element of $S - \{x_0, \dots, x_{n-1}\}$. In this way we obtain an infinite listing of the elements of S :

$$x_0, x_1, x_2, \dots, x_n, \dots$$

Notice that each element $m \in S$ is in the listing because there are at most m elements of S that are less than m . So m must be represented as one of the elements $x_0, x_1, x_2, \dots, x_m$ in the sequence. The association x_k to k gives a bijection between S and $\mathbb{N}$. So $|S| = |\mathbb{N}|$ and thus S is countable.

(b) If S is countable then $|S| \leq |\mathbb{N}|$ by definition. So assume that $|S| \leq |\mathbb{N}|$. Then there is an injection $f: S \rightarrow \mathbb{N}$. So $|S| = |f(S)|$. Since $f(S)$ is a subset of $\mathbb{N}$, it is countable by (a). Therefore $f(S)$ is either finite or $|f(S)| = |\mathbb{N}|$. So S is either finite or $|S| = |f(S)| = |\mathbb{N}|$. QED

Techniques to Show Countability

An interesting and useful fact about countability is that the set $\mathbb{N} \times \mathbb{N}$ is countable. We'll state it for the record.

Theorem

(2.10)

$\mathbb{N} \times \mathbb{N}$ is a countable set.

Proof: We need to describe a bijection between $\mathbb{N} \times \mathbb{N}$ and $\mathbb{N}$. We'll do this by arranging the elements of $\mathbb{N} \times \mathbb{N}$ in such a way that they can be easily counted. One way to do this is shown in the following listing, where each row lists a sequence of tuples in $\mathbb{N} \times \mathbb{N}$ followed by a corresponding sequence of natural numbers.

$$\begin{array}{ccc}
 (0, 0), & \longleftrightarrow & 0, \\
 (0, 1), (1, 0), & \longleftrightarrow & 1, 2 \\
 (0, 2), (1, 1), (2, 0), & \longleftrightarrow & 3, 4, 5, \\
 \vdots & \vdots & \vdots \\
 (0, n), \dots & \longleftrightarrow & (n^2 + n)/2, \dots \\
 \vdots & \vdots & \vdots
 \end{array}$$

Notice that each row of the listing contains all the tuples whose components add up to the same number. For example, the sequence $(0, 2), (1, 1), (2, 0)$ consists of all tuples whose components add up to 2. So we have a bijection between $\mathbb{N} \times \mathbb{N}$ and $\mathbb{N}$. Therefore, $\mathbb{N} \times \mathbb{N}$ is countable. QED.

We should note that the bijection described in (2.10) is called *Cantor's pairing function*. It maps each pair of natural numbers (x, y) to the natural number

$$\frac{(x + y)^2 + 3x + y}{2}.$$

We can use (2.10) to prove the following result that the union of a countable collection of countable sets is countable.

Counting Unions of Countable Sets

(2.11)

If $S_0, S_1, \dots, S_n, \dots$ is a sequence of countable sets, then the union

$$S_0 \cup S_1 \cup \dots \cup S_n \cup \dots$$

is a countable set.

Proof: Since each set is countable, its elements can be indexed by natural numbers. So for each set S_n we'll list its elements as $x_{n0}, x_{n1}, x_{n2}, \dots$. If S_n is a finite set then we'll list one of its elements repeatedly to make the listing infinite. In the same way, if there are only finitely many sets, then we'll list one of the sets repeatedly to make the sequence infinite. In this way we can associate each tuple (m, n) in $\mathbb{N} \times \mathbb{N}$ with an element x_{mn} in the union of the given sets. The mapping may not be a bijection since some elements of the union might be repeated in the listings. But the mapping is a surjection from $\mathbb{N} \times \mathbb{N}$ to the union of the sets. So, since $\mathbb{N} \times \mathbb{N}$ is countable, it follows from (2.9d) that the union is countable. QED.

example 2.26 Countability of the Rationals

We'll show that the set $\mathbb{Q}$ of rational numbers is countable by showing that $|\mathbb{Q}| = |\mathbb{N}|$. Let $\mathbb{Q}^+$ denote the set of positive rational numbers. So we can represent $\mathbb{Q}^+$ as the following set of fractions, where repetitions are included (e.g., $1/1$ and $2/2$ are both elements of the set).

$$\mathbb{Q}^+ = \{m/n \mid m, n \in \mathbb{N} \text{ and } n \neq 0\}.$$

Now we'll associate each fraction m/n with the tuple (m, n) in $\mathbb{N} \times \mathbb{N}$. This association is an injection, so we have $|\mathbb{Q}^+| \leq |\mathbb{N} \times \mathbb{N}|$. Since $\mathbb{N} \times \mathbb{N}$ is

countable, it follows that $\mathbb{Q}^+$ is countable. In the same way, the set $\mathbb{Q}^-$ of negative rational numbers is countable. Now we can write $\mathbb{Q}$ as the union of three countable sets:

$$\mathbb{Q} = \mathbb{Q}^+ \cup \{0\} \cup \mathbb{Q}^-.$$

Since each set in the union is countable, it follows from (2.11) that the union of the sets is countable.

end example

Counting Strings

An important consequence of (2.11) is the following fact about the countability of the set of all strings over a finite alphabet.

Theorem

(2.12)

The set A^* of all strings over a finite alphabet A is countably infinite.

Proof: For each $n \in \mathbb{N}$, let A_n be the set of strings over A having length n . It follows that A^* is the union of the sets $A_0, A_1, \dots, A_n, \dots$. Since each set A_n is finite, we can apply (2.11) to conclude that A^* is countable. QED.

2.4.3 Diagonalization

Let's discuss a classic construction technique, called *diagonalization*, which is quite useful in several different settings that deal with counting. The technique was introduced by Cantor when he showed that the real numbers are uncountable. Here is a general description of diagonalization.

Diagonalization

(2.13)

Let A be an alphabet with two or more symbols and let $S_0, S_1, \dots, S_n, \dots,$

be a countable listing of sequences of the form $S_n = (a_{n0}, a_{n1}, \dots, a_{nn}, \dots)$, where $a_{ni} \in A$. The sequences are listed as the rows of the following infinite matrix.

	0	1	2	$\dots$	n	$\dots$
S_0	a_{00}	a_{01}	a_{02}	$\dots$	a_{0n}	$\dots$
S_1	a_{10}	a_{11}	a_{12}	$\dots$	a_{1n}	$\dots$
S_2	a_{20}	a_{21}	a_{22}	$\dots$	a_{2n}	$\dots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$
S_n	a_{n0}	a_{n1}	a_{n2}	$\dots$	a_{nn}	$\dots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

Then there is a sequence $S = (a_0, a_1, a_2, \dots, a_n, \dots)$ over A that is *not* in the original list. We can construct S from the list of diagonal elements $(a_{00}, a_{11}, a_{22}, \dots, a_{nn}, \dots)$ by changing each element so that $a_n \neq a_{nn}$ for each n . Therefore, S differs from each S_n at the n th element. For example, pick two elements $x, y \in A$ and define

$$a_n = \begin{cases} x & \text{if } a_{nn} = y \\ y & \text{if } a_{nn} \neq y. \end{cases}$$

Uncountable Sets

Now we're in position to give some examples of uncountable sets. We'll demonstrate the method of Cantor, which uses proof by contradiction together with the diagonalization technique.

example 2.27 Uncountability of the Reals

We'll show that the set $\mathbb{R}$ of real numbers is uncountable. It was shown in Exercise 6e of Section 2.3 that there is a bijection between R and the set U of real numbers between 0 and 1. So $|\mathbb{R}| = |U|$ and we need only show that U is uncountable. Assume, by way of contradiction, that U is countable. Then we can list all the numbers between 0 and 1 as a countable sequence

$$r_0, r_1, r_2, \dots, r_n, \dots$$

Each real number between 0 and 1 can be represented as an infinite decimal. So for each n there is a representation $r_n = 0.d_{n0}d_{n1} \dots d_{nn} \dots$, where each d_{ni} is a decimal digit. Since we can also represent r_n by the sequence of decimal digits $(d_{n0}, d_{n1}, \dots, d_{nn}, \dots)$, it follows by diagonalization (2.13) that there is an infinite decimal that is not in the list. For example, we'll choose the digits 1 and 2 to construct the number $s = 0.s_0s_1s_2 \dots$ where

$$s_k = \begin{cases} 1 & \text{if } d_{kk} = 2 \\ 2 & \text{if } d_{kk} \neq 2. \end{cases}$$

So $0 < s < 1$ and s differs from each r_n at the n th decimal place. Thus s is not in the list, contrary to our assumption that we have listed all numbers in U . So the set U is uncountable, and hence $\mathbb{R}$ is also uncountable.

end example

example 2.28 Natural Number Functions

How many different functions are there from $\mathbb{N}$ to $\mathbb{N}$? We'll show that the set of all such functions is uncountable. Assume, by way of contradiction, that the set is countable. Then we can list all the functions type $\mathbb{N} \rightarrow \mathbb{N}$ as $f_0, f_1, \dots, f_n, \dots$. We'll represent each function f_n as the sequence of its values $(f_n(0), f_n(1), \dots, f_n(n), \dots)$. Now (2.13) tells us there is a function missing from the list, which contradicts our assumption that all functions are in the list. So the set of all functions of type $\mathbb{N} \rightarrow \mathbb{N}$ is uncountable.

For example, we might choose the numbers $1, 2 \in \mathbb{N}$ and define a function $g : \mathbb{N} \rightarrow \mathbb{N}$ by

$$g(n) = \begin{cases} 1 & \text{if } f_n(n) = 2 \\ 2 & \text{if } f_n(n) \neq 2. \end{cases}$$

Then the sequence of values $(g(0), g(1), \dots, g(n), \dots)$ is different from each of the sequences for the listed functions because $g(n) \neq f_n(n)$ for each n . There are many different ways to find a function that it is not in the list. For example, we could define g by

$$g(n) = f_n(n) + 1.$$

It follows that $g(n) \neq f_n(n)$ for each n . So g cannot be in the list $f_0, f_1, \dots, f_n, \dots$

end example

2.4.4 Limits on Computability

Let's have a short discussion about whether there are limits on what can be computed. As another application of (2.12) we can answer the question: How many programs can be written in your favorite programming language? The answer is countably infinite. Here is the result.

Theorem

(2.14)

The set of programs for a programming language is countably infinite.

Proof: One way to see this is to consider each program as a finite string of symbols over a fixed finite alphabet A . For example, A might consist of all characters that can be typed from a keyboard. Now we can proceed as in the proof of (2.12). For each natural number n , let P_n denote the set of all programs that are strings of length n over A . For example, the program

```
{print('help')}
```

is in P_{15} because it's a string of length 15. So the set of all programs is the union of the sets $P_0, P_1, \dots, P_n, \dots$. Since each P_n is finite, hence countable, it follows from (2.11) that the union is countable. QED

Not Everything Is Computable

Since there are “only” a countable number of computer programs, it follows that there are limits on what can be computed. For example, there are an uncountable number of functions of type $\mathbb{N} \rightarrow \mathbb{N}$. So there are programs to calculate only a countable subset of these functions.

Can any real number be computed to any given number of decimal places? The answer is no. The reason is that there are “only” a countable number of computer programs (2.14) but the set of real numbers is uncountable. Therefore, there are only a countable number of computable numbers in $\mathbb{R}$ because each computable number needs a program to compute it. If we remove the computable numbers from $\mathbb{R}$, the resulting set is still uncountable. Can you see why? So most real numbers cannot be computed.

The rational numbers can be computed, and there are also many irrational numbers that can be computed. Pi is the most famous example of a computable irrational number. In fact, there are countably infinitely many computable irrational numbers.

Higher Cardinalities

It's easy to find infinite sets having many different cardinalities because Cantor proved that there are more subsets of a set than there are elements of the set. In other words, for any set A , we have the following result.

Theorem

(2.15)

$$|A| < |\text{power}(A)|.$$

We know this is true for finite sets. But it's also true for infinite sets. We'll discuss the proof in an exercise. Notice that if A is countably infinite, then we can conclude from (2.15) that $\text{power}(A)$ is uncountable. So, for example, we

can conclude that $\text{power}(\mathbb{N})$ is uncountable.

For another example, we might wonder how many different languages there are over a finite alphabet such as $\{a, b\}$. Since a language over $\{a, b\}$ is a set of strings over $\{a, b\}$, it follows that such a language is a subset of $\{a, b\}^*$, the set of all strings over $\{a, b\}$. So the set of all languages over $\{a, b\}$ is $\text{power}(\{a, b\}^*)$. From (2.12) we can conclude that $\{a, b\}^*$ is countably infinite. In other words, we have $|\{a, b\}^*| = |\mathbb{N}|$. So we can use (2.15) to obtain

$$|\mathbb{N}| = |\{a, b\}^*| < |\text{power}(\{a, b\}^*)|.$$

Therefore, $\text{power}(\{a, b\}^*)$ is uncountable, which is the same as saying that there are uncountably many languages over the alphabet $\{a, b\}$. Of course, this generalizes to any finite alphabet. So we have the following statement.

Theorem

(2.16)

There are uncountably many languages over a finite alphabet.

We can use (2.15) to find infinite sequences of sets of higher and higher cardinality. For example, we have

$$|\mathbb{N}| < |\text{power}(\mathbb{N})| < |\text{power}(\text{power}(\mathbb{N}))| < \cdots$$

Can we associate these sets with more familiar sets? Sure, it can be shown that $|\mathbb{R}| = |\text{power}(\mathbb{N})|$, which we'll discuss in an exercise. So we have

$$|\mathbb{N}| < |\mathbb{R}| < |\text{power}(\text{power}(\mathbb{N}))| < \cdots$$

Is there any “well-known” set S such that $|S| = |\text{power}(\text{power}(\mathbb{N}))|$? Since the real numbers are hard enough to imagine, how can we comprehend all the elements in $\text{power}(\text{power}(\mathbb{N}))$? Luckily, in computer science we will seldom, if ever, have occasion to worry about sets having higher cardinality than the set of real numbers.

The Continuum Hypothesis

We'll close the discussion with a question: Is there a set S whose cardinality is between that of $\mathbb{N}$ and that of the real numbers $\mathbb{R}$? In other words, does there

exist a set S such that $|\mathbb{N}| < |S| < |\mathbb{R}|$? The answer is that no one knows. Interestingly, it has been shown that people who assume that the answer is yes won't run into any contradictions by using the assumption in their reasoning. Similarly, it has been shown that people who assume that the answer is no won't run into any contradictions by using the assumption in their arguments! The assumption that the answer is no is called the *continuum hypothesis*.

If we accept the continuum hypothesis, then we can use it as part of a proof technique. For example, suppose that for some set S we can show that $|\mathbb{N}| \leq |S| < |\mathbb{R}|$. Then we can conclude that $|\mathbb{N}| = |S|$ by the continuum hypothesis.

Exercises

Finite Sets

1. Find the cardinality of each set by establishing a bijection between it and a set of the form $\{0, 1, \dots, n\}$.
 - a. $\{2x + 5 \mid x \in \mathbb{N} \text{ and } 1 \leq 2x + 5 \leq 98\}$.
 - b. $\{x^2 \mid x \in \mathbb{N} \text{ and } 0 \leq x^2 \leq 500\}$.
 - c. $\{2, 5, 8, 11, 14, 17, \dots, 44, 47\}$.

Countable Infinite Sets

2. Show that each of the following sets is countable by establishing a bijection between the set and $\mathbb{N}$.
 - a. The set of even natural numbers.
 - b. The set of negative integers.
 - c. The set of strings over $\{a\}$.
 - d. The set of lists over $\{a\}$ that have even length.
 - e. The set $\mathbb{Z}$ of integers.
 - f. The set of odd integers.
 - g. The set of even integers.
3. Use (2.11) to show that each of the following sets is countable by describing the set as a union of countable sets.

- a. The set of strings over $\{a, b\}$ that have odd length.
- b. The set of all lists over $\{a, b\}$.
- c. The set of all binary trees over $\{a, b\}$.
- d. $\mathbb{N} \times \mathbb{N} \times \mathbb{N}$.

Diagonalization

4. For each countable set of infinite sequences, use diagonalization (2.13) to construct an infinite sequence of the same type that is not in the set.
 - a. $\{(f_n(0), f_n(1), \dots, f_n(n), \dots) \mid f_n(k) \in \{\text{hello, world}\} \text{ for } n, k \in \mathbb{N}\}$.
 - b. $\{(f(n, 0), f(n, 1), \dots, f(n, n), \dots) \mid f(n, k) \in \{a, b, c\} \text{ for } n, k \in \mathbb{N}\}$.
 - c. $\{(a_n0, a_n1, \dots, a_nn, \dots) \mid a_{nk} \in \{2, 4, 6, 8\} \text{ for } n, k \in \mathbb{N}\}$.
5. To show that $\text{power}(\mathbb{N})$ is uncountable, we can proceed by way of contradiction. Assume that it is countable, so that all the subsets of $\mathbb{N}$ can be listed $S_0, S_1, S_2, \dots, S_n, \dots$. Complete the proof by finding a way to represent each subset of $\mathbb{N}$ as an infinite sequence of 1's and 0's, where 1 means true and 0 means false. Then a contradiction arises by using diagonalization (2.13) to construct an infinite sequence of the same type that represents a subset of $\mathbb{N}$ that is not listed.

Proofs and Challenges

6. Show that if A is uncountable and B is a countable subset of A , then the set $A - B$ is uncountable.
7. Prove each statement about countable sets:
 - a. Any subset of a countable set is countable.
 - b. Any image of a countable set is countable.
8. Let A be a countably infinite alphabet $A = \{a_0, a_1, a_2, \dots\}$. Let A^* denote the set of all strings over A . For each $n \in \mathbb{N}$, let A_n denote the set of all strings in A^* having length n .
 - a. Show that A_n is countable for $n \in \mathbb{N}$. *Hint:* Use (2.11).
 - b. Show that A^* is countable. *Hint:* Use (2.11) and Part (a).

9. Let $\text{finite}(\mathbb{N})$ denote the set of all finite subsets of $\mathbb{N}$. Use (2.11) to show that $\text{finite}(\mathbb{N})$ is countable.
10. We'll start a proof that $|A| < |\text{power}(A)|$ for any set A . Proof: Since each element $x \in A$ can be associated with $\{x\} \in \text{power}(A)$, it follows that $|A| \leq |\text{power}(A)|$. To show that $|A| < |\text{power}(A)|$ we'll assume, by way of contradiction, that there is a bijection $A \rightarrow \text{power}(A)$. So each $x \in A$ is associated with a subset S_x of A . Now, define the following subset of A .

$$S = \{x \in A \mid x \notin S_x\}.$$

Since S is a subset of A , our assumed bijection tells us that there must be an element y in A that is associated with S . In other words, $S_y = S$. Find a contradiction by observing where y is and where it is not.

2.5 Chapter Summary

Functions allow us to associate different sets of objects. They are characterized by associating each domain element with a unique codomain element. For any function $f: A \rightarrow B$, subsets of the domain A have images in the codomain B and subsets of B have pre-images in A . The image of A is the range of f . Partial functions need not be defined for all domain elements.

Some functions that are particularly useful in computer science are floor, ceiling, greatest common divisor (with the associated division algorithm), mod, and log.

Composition is a powerful tool for constructing new functions from known functions. Three functions that are useful in programming with lists are sequence, distribute, and pairs. The map function is a useful tool for computing lists of values of a function.

Three important properties of functions that allow us to compare sets are injective, surjective, and bijective. These properties are useful in describing the pigeonhole principle and in working with ciphers and hash functions. These properties are also useful in comparing the cardinality of sets.

A set is countable if it is finite or has the same cardinality as the set of natural numbers. Countable unions of countable sets are countable. The set of all computer programs is countable. The diagonalization technique can be used

to show that some countable listings are not exhaustive. It can also be used to show that some sets, such as the real numbers, are uncountable. So we can't compute all the real numbers. Any set has smaller cardinality than its power set, even when the set is infinite.

¹Leibniz introduced the word “function” around 1692. He is responsible for such diverse ideas as binary arithmetic, symbolic logic, combinatorics, and calculus. Around 1694 he built a calculating machine that could add and multiply.

Construction Techniques

When we build, let us think that we build forever.

—John Ruskin (1819–1900)

To construct an object, we need some kind of description. If we're lucky, the description might include a construction technique. Otherwise, we may need to use our wits and our experience to construct the object. This chapter focuses on gaining some construction experience.

The only way to learn a technique is to use it on a wide variety of problems. We'll present each technique in the framework of objects that occur in computer science, and as we go along, we'll extend our knowledge of these objects. We'll begin by introducing the technique of inductive definition for sets. Then we'll discuss techniques for describing recursively defined functions and procedures. Last but not least, we'll introduce grammars for describing sets of strings.

There are usually two parts to solving a problem. The first is to guess at a solution and the second is to verify that the guess is correct. The focus of this chapter is to introduce techniques to help us make good guesses. We'll usually check a few cases to satisfy ourselves that our guesses are correct. In the next chapter we'll study inductive proof techniques that can be used to actually prove correctness of claims about objects constructed by the techniques of this chapter.

chapter guide

3.1 (Inductively Defined Sets) introduces the inductive definition technique. We'll apply the technique by defining various sets of numbers, strings,

lists, binary trees, and Cartesian products.

- 3.2 (Recursive Functions and Procedures) introduces the technique of recursive definition for functions and procedures. We'll apply the technique to functions and procedures that process numbers, strings, lists, and binary trees. We'll solve the repeated element problem and the power set problem, and we'll construct some functions for infinite sequences.
- 3.3 (Grammars) introduces the idea of a grammar as a way to describe a language. We'll see that grammars describe the strings of a language in an inductive fashion, and we'll see that they provide recursive rules for testing whether a string belongs to a language.

3.1 Inductively Defined Sets

When we write down an informal statement such as $A = \{3, 5, 7, 9, \dots\}$, most of us will agree that we mean the set $A = \{2k + 3 \mid k \in \mathbb{N}\}$. Another way to describe A is to observe that $3 \in A$, that $x \in A$ implies $x + 2 \in A$, and that the only way an element gets in A is by the previous two steps. This description has three ingredients, which we'll state informally as follows:

1. There is a starting element (3 in this case).
2. There is a construction operation to build new elements from existing elements (addition by 2 in this case).
3. There is a statement that no other elements are in the set.

Definition of Inductive Definition

This process is an example of an *inductive definition* of a set. The set of objects defined is called an *inductive set*. An inductive set consists of objects that are constructed, in some way, from objects that are already in the set. So nothing can be constructed unless there is at least one object in the set to start the process. Inductive sets are important in computer science because the objects can be used to represent information and the construction rules can often be programmed. We give the following formal definition.

An inductive definition of a set S consists of three steps:

(3.1)

Basis: Specify one or more elements of S .

Induction: Give one or more rules to construct new elements of S from existing elements of S .

Closure: State that S consists exactly of the elements obtained by the basis and induction steps. This step is usually assumed rather than stated explicitly.

The closure step is a very important part of the definition. Without it, there could be lots of sets satisfying the first two steps of an inductive definition. For example, the two sets $\mathbb{N}$ and $\{3, 5, 7, \dots\}$ both contain the number 3, and if x is in either set, then so is $x + 2$. It's the closure statement that tells us that the only set defined by the basis and induction steps is $\{3, 5, 7, \dots\}$. So the closure statement tells us that we're defining exactly one set, namely, the smallest set satisfying the basis and induction steps. We'll always omit the specific mention of closure in our inductive definitions.

The *constructors* of an inductive set are the basis elements and the rules for constructing new elements. For example, the inductive set $\{3, 5, 7, 9, \dots\}$ has two constructors, the number 3 and the operation of adding 2 to a number.

For the rest of this section we'll use the technique of inductive definition to construct sets of objects that are often used in computer science.

3.1.1 Numbers

The set of natural numbers $\mathbb{N} = \{0, 1, 2, \dots\}$ is an inductive set. Its basis element is 0, and we can construct a new element from an existing one by adding the number 1. So we can write an inductive definition for $\mathbb{N}$ in the following way.

Basis: $0 \in \mathbb{N}$.

Induction: If $n \in \mathbb{N}$, then $n + 1 \in \mathbb{N}$.

The constructors of $\mathbb{N}$ are the integer 0 and the operation that adds 1 to an element of $\mathbb{N}$. The operation of adding 1 to n is called the *successor* function, which we write as

$$\text{succ}(n) = n + 1.$$

Using the successor function, we can rewrite the induction step in the above definition of $\mathbb{N}$ in the alternative form

$$\text{If } n \in \mathbb{N}, \text{ then } \text{succ}(n) \in \mathbb{N}.$$

So we can say that $\mathbb{N}$ is an inductive set with two constructors, 0 and succ.

example 3.1 Some Familiar Odd Numbers

We'll give an inductive definition of $A = \{1, 3, 7, 15, 31, \dots\}$. Of course, the basis case should place 1 in A . If $x \in A$, then we can construct another element of A with the expression $2x + 1$. So the constructors of A are the number 1 and the operation of multiplying by 2 and adding 1. An inductive definition of A can be written as follows:

Basis: $1 \in A$.

Induction: If $x \in A$, then $2x + 1 \in A$.

example 3.2 Some Even and Odd Numbers

Is the following set inductive?

$$A = \{2, 3, 4, 7, 8, 11, 15, 16, \dots\}.$$

It might be easier if we think of A as the union of the two sets

$$B = \{2, 4, 8, 16, \dots\} \text{ and } C = \{3, 7, 11, 15, \dots\}.$$

Both of these sets are inductive. The constructors of B are the number 2 and the operation of multiplying by 2. The constructors of C are the number 3 and the operation of adding by 4. We can combine these definitions to give an inductive definition of A .

Basis: $2, 3 \in A$.

Induction: If $x \in A$ and x is odd, then $x + 4 \in A$.

If $x \in A$ and x is even, then $2x \in A$.

This example shows that there can be more than one basis element, more than one induction rule, and tests can be included.

end example

example 3.3 Communicating with a Robot

Suppose we want to communicate the idea of the natural numbers to a robot that knows about functions, has a loose notion of sets, and can follow an inductive definition. Symbols like 0 , 1 , $\dots$, and $+$ make no sense to the robot. How can we convey the idea of $\mathbb{N}$? We'll tell the robot that N is the name of the set we want to construct.

Suppose we start by telling the robot to put the symbol 0 in N . For the induction case we need to tell the robot about the successor function. We tell the robot that $s : N \rightarrow N$ is a function, and whenever an element $x \in N$, then put the element $s(x) \in N$. After a pause, the robot says, " $N = \{0\}$ because I'm letting s be the function defined by $s(0) = 0$."

Since we don't want $s(0) = 0$, we have to tell the robot that $s(0) \neq 0$. Then the robot says, " $N = \{0, s(0)\}$ because $s(s(0)) = 0$." So we tell the robot that $s(s(0)) \neq 0$. Since this could go on forever, let's tell the robot that $s(x)$ does not equal any previously defined element. Do we have it? Yes. The robot responds with " $N = \{0, s(0), s(s(0)), s(s(s(0))), \dots\}$." So we can give the robot the following definition:

Basis: $0 \in N$.

Induction: If $x \in N$, then put $s(x) \in N$, where $s(x) \neq 0$ and $s(x)$ is not equal to any previously defined element of N .

This definition of the natural numbers—along with a closure statement—is due to the mathematician and logician Giuseppe Peano (1858–1932).

example 3.4 Communicating with Another Robot

Suppose we want to define the natural numbers for a robot that knows about sets and can follow an inductive definition. How can we convey the idea of $\mathbb{N}$ to the robot? Since we can use only the notation of sets, let's use $\emptyset$ to stand for the number 0.

What about the number 1? Can we somehow convey the idea of 1 using the empty set? Let's let $\{\emptyset\}$ stand for 1. What about 2? We can't use $\{\emptyset, \emptyset\}$, because $\{\emptyset, \emptyset\} = \{\emptyset\}$. Let's let $\{\emptyset, \{\emptyset\}\}$ stand for 2 because it has two distinct elements. Notice the little pattern we have going: If s is the set standing for a number, then $s \cup \{s\}$ stands for the successor of the number.

Starting with $\emptyset$ as the basis element, we have an inductive definition. Letting Nat be the set that we are defining for the robot, we have the following inductive definition.

Basis: $\emptyset \in Nat$.

Induction: If $s \in Nat$, then $s \cup \{s\} \in Nat$.

For example, since 2 is represented by the set $\{\emptyset, \{\emptyset\}\}$, the number 3 is represented by the set

$$\{\emptyset, \{\emptyset\}\} \cup \{\{\emptyset, \{\emptyset\}\}\} = \{\emptyset, \{\emptyset\}, \{\emptyset, \{\emptyset\}\}\}.$$

This is not fun. After a while we might try to introduce some of our own notation to the robot. For example, we'll introduce the decimal numerals in the following way.

$$\begin{aligned} 0 &= \emptyset, \\ 1 &= 0 \cup \{0\}, \\ 2 &= 1 \cup \{1\}, \\ &\vdots \end{aligned}$$

Now we can think about the natural numbers in the following way.

$$\begin{aligned} 0 &= \emptyset, \\ 1 &= 0 \cup \{0\} = \emptyset \cup \{0\} = \{0\}, \\ 2 &= 1 \cup \{1\} = \{0\} \cup \{1\} = \{0, 1\}, \\ &\vdots \end{aligned}$$

Therefore, each number is the set of numbers that precedes it.

end example

3.1.2 Strings

We often define strings of things inductively without even thinking about it. For example, in high school algebra we might say that an algebraic expression is either a number or a variable, and if A and B are algebraic expressions, then so are (A) , $A + B$, $A - B$, AB , and $A \div B$. So the set of algebraic expressions is a set of strings. For example, if x and y are variables, then the following strings are algebraic expressions.

$$x, \quad y, \quad 25, \quad 25x, \quad x + y, \quad (4x + 5y), \quad (x + y)(2yx), \quad 3x \div 4.$$

If we like, we can make our definition more formal by specifying the basis and induction parts. For example, if we let E denote the set of algebraic expressions as we have described them, then we have the following inductive definition for E .

Basis: If x is a variable or a number, then $x \in E$.

Induction: If $A, B \in E$, then (A) , $A + B$, $A - B$, AB , $A \div B \in E$.

Let's recall that for an alphabet A , the set of all strings over A is denoted by A^* . This set has the following inductive definition.



All Strings over A

(3.2)

Basis: $\Lambda \in A^*$.

Induction: If $s \in A^*$ and $a \in A$, then $as \in A^*$.

We should note that when we place two strings next to each other in juxtaposition to form a new string, we are concatenating the two strings. So, from a computational point of view, concatenation is the operation we are using to construct new strings.

Recall that any set of strings is called a *language*. If A is an alphabet, then any language over A is one of the subsets of A^* . Many languages can be defined inductively. Here are some examples.

example 3.5 Three Languages

We'll give an inductive definition for each of three languages.

$$1. S = \{a, ab, abb, abbb, \dots\} = \{ab^n \mid n \in \mathbb{N}\}.$$

Informally, we can say that the strings of S consist of the letter a followed by zero or more b 's. But we can also say that the letter a is in S , and if x is a string in S , then so is xb . This gives us an inductive definition for S .

Basis: $a \in S$.

Induction: If $x \in S$, then $xb \in S$.

$$2. S = \{\Lambda, ab, aabb, aaabbb, \dots\} = \{a^n b^n \mid n \in \mathbb{N}\}.$$

Informally, we can say that the strings of S consist of any number of a 's followed by the same number of b 's. But we can also say that the empty string Λ is in S , and if x is a string in S , then so is axb . This gives us an inductive definition for S .

Basis: $\Lambda \in S$.

Induction: If $x \in S$, then $axb \in S$.

$$3. S = \{\Lambda, ab, abab, ababab, \dots\} = \{(ab)^n \mid n \in \mathbb{N}\}.$$

Informally, we can say that the strings of S consist of any number of ab pairs. But we can also say that the empty string Λ is in S , and if x is a string in S , then so is abx . This gives us an inductive definition for S .

Basis: $\Lambda \in S$.

Induction: If $x \in S$, then $abx \in S$.

end example

example 3.6 Decimal Numerals

Let's give an inductive definition for the set of decimal numerals. Recall that a decimal numeral is a nonempty string of decimal digits. For example, 2340 and 002965 are decimal numerals. If we let D denote the set of decimal numerals, we can describe D by saying that any decimal digit is in D , and if x is in D and d is a decimal digit, then dx is in D . This gives us the following inductive definition for D :

Basis: $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\} \subseteq D$.

Induction: If $x \in D$ and d is a decimal digit, then $dx \in D$.

end example

3.1.3 Lists

Recall that a list is an ordered sequence of elements. Let's try to find an inductive definition for the set of lists with elements from a set A . In [Chapter 1](#) we denoted the set of all lists over A by $\text{lists}(A)$, and we'll continue to do so. We also mentioned that from a computational point of view the only parts of a nonempty list that can be accessed randomly are its *head* and its *tail*. Head and tail are sometimes called *destructors*, since they are used to destroy a list (take it apart). For example, the list $\langle x, y, z \rangle$ has x as its head and $\langle y, z \rangle$ as its

tail, which we write as

$$\text{head}(\langle x, y, z \rangle) = x \quad \text{and} \quad \text{tail}(\langle x, y, z \rangle) = \langle y, z \rangle.$$

We also introduced the operation “cons” to construct lists, where if h is an element and t is a list, the new list whose head is h and whose tail is t is represented by the expression

$$\text{cons}(h, t).$$

So cons is a constructor of lists. For example, we have

$$\text{cons}(x, \langle y, z \rangle) = \langle x, y, z \rangle$$

$$\text{cons}(x, \langle \rangle) = \langle x \rangle.$$

The operations cons, head, and tail work nicely together. For example, we can write

$$\langle x, y, z \rangle = \text{cons}(x, \langle y, z \rangle) = \text{cons}(\text{head}(\langle x, y, z \rangle), \text{tail}(\langle x, y, z \rangle)).$$

So if L is any nonempty list, then we have the equation

$$L = \text{cons}(\text{head}(L), \text{tail}(L)).$$

Now we have the proper tools, so let's get down to business and write an inductive definition for $\text{lists}(A)$. Informally, we can say that $\text{lists}(A)$ is the set of all ordered sequences of elements taken from the set A . But we can also say that $\langle \rangle$ is in $\text{lists}(A)$, and if L is in $\text{lists}(A)$, then so is $\text{cons}(a, L)$ for any a in A . This gives us an inductive definition for $\text{lists}(A)$, which we can state formally as follows.

All Lists over A

(3.3)

Basis: $\langle \rangle \in \text{lists}(A).$

Induction:

If $x \in A$ and $L \in \text{lists}(A)$, then $\text{cons}(x, L) \in \text{lists}(A)$.

example 3.7 List Membership

Let $A = \{a, b\}$. We'll use (3.3) to show how some lists become members of $\text{lists}(A)$. The basis case puts $\langle \rangle \in \text{lists}(A)$. Since $a \in A$ and $\langle \rangle \in \text{lists}(A)$, the induction step gives

$$\langle a \rangle = \text{cons}(a, \langle \rangle) \in \text{lists}(A).$$

In the same way we get $\langle b \rangle \in \text{lists}(A)$. Now since $a \in A$ and $\langle a \rangle \in \text{lists}(A)$, the induction step puts $\langle a, a \rangle \in \text{lists}(A)$. Similarly, we get $\langle b, a \rangle$, $\langle a, b \rangle$, and $\langle b, b \rangle$ as elements of $\text{lists}(A)$, and so on.

end example

A Notational Convenience

It's convenient when working with lists to use an infix notation for cons to simplify the notation for list expressions. We'll use the double colon symbol $::$, so that the infix form of $\text{cons}(x, L)$ is

$$x :: L.$$

For example, the list $\langle a, b, c \rangle$ can be constructed using cons as

$$\begin{aligned} \text{cons}(a, \text{cons}(b, \text{cons}(c, \langle \rangle))) &= \text{cons}(a, \text{cons}(b, \langle c \rangle)) \\ &= \text{cons}(a, \langle b, c \rangle) \\ &= \langle a, b, c \rangle. \end{aligned}$$

Using the infix form, we construct $\langle a, b, c \rangle$ as follows:

$$a :: (b :: (c :: \langle \rangle)) = a :: (b :: \langle c \rangle) = a :: \langle b, c \rangle = \langle a, b, c \rangle.$$

The infix form of cons allows us to omit parentheses by agreeing that $::$ is right associative. In other words, $a :: b :: L = a :: (b :: L)$. Thus we can represent the

list $\langle a, b, c \rangle$ by writing

$a :: b :: c :: \langle \rangle$ instead of $a :: (b :: (c :: \langle \rangle))$.

Many programming problems involve processing data represented by lists. The operations `cons`, `head`, and `tail` provide basic tools for writing programs to create and manipulate lists. So they are necessary for programmers. Now let's look at a few examples.

example 3.8 Lists of Binary Digits

Suppose we need to define the set S of all nonempty lists over the set $\{0, 1\}$ with the property that adjacent elements in each list are distinct. We can get an idea about S by listing a few elements:

$$S = \{\langle 0 \rangle, \langle 1 \rangle, \langle 1, 0 \rangle, \langle 0, 1 \rangle, \langle 0, 1, 0 \rangle, \langle 1, 0, 1 \rangle, \dots\}.$$

Let's try `0` and `1` as basis elements of S . Then we can construct a new list from a list $L \in S$ by testing whether `head(L)` is 0 or 1. If `head(L) = 0`, then we place 1 at the left of L . Otherwise, we place 0 at the left of L . So we can write the following inductive definition for S .

Basis: $0, 1 \in S$.

Induction: If $L \in S$ and `head(L) = 0`, then `cons(1, L) ∈ S`.

If $L \in S$ and `head(L) = 1`, then `cons(0, L) ∈ S`.

The infix form of these induction rules looks like

If $L \in S$ and `head(L) = 0`, then $1 :: L \in S$.

If $L \in S$ and `head(L) = 1`, then $0 :: L \in S$.

end example

example 3.9 Lists of Letters

Suppose we need to define the set S of all lists over $\{a, b\}$ that begin with the single letter a followed by zero or more occurrences of b . We can describe S informally by writing a few of its elements:

$$S = \{\langle a \rangle, \langle a, b \rangle, \langle a, b, b \rangle, \langle a, b, b, b \rangle, \dots\}.$$

It seems appropriate to make $\langle a \rangle$ the basis element of S . Then we can construct a new list from any list $L \in S$ by attaching the letter b on the right end of L . But cons places new elements at the left end of a list. We can overcome the problem in the following way:

$$\text{If } L \in S, \text{ then } \text{cons}(a, \text{cons}(b, \text{tail}(L))) \in S.$$

In infix form the statement reads as follows:

$$\text{If } L \in S, \text{ then } a :: b :: \text{tail}(L) \in S.$$

For example, if $L = \langle a \rangle$, then we construct the list

$$a :: b :: \text{tail}(\langle a \rangle) = a :: b :: \langle \rangle = a :: \langle b \rangle = \langle a, b \rangle.$$

So we have the following inductive definition of S :

Basis: $\langle a \rangle \in S.$

Induction: If $L \in S$, then $a :: b :: \text{tail}(L) \in S.$

end example

example 3.10 All Possible Lists

Can we find an inductive definition for the set of all possible lists over $\{a, b\}$, including lists that can contain other lists? Suppose we start with lists having a small number of symbols, including the symbols $\langle$ and $\rangle$. Then, for each $n \geq 2$, we can write down the lists made up of n symbols (not including commas). Figure 3.1 shows these listings for the first few values of n .

If we start with the empty list $\langle \rangle$, then with a and b we can construct three more lists as follows:

$$\begin{aligned}
a &:: \langle \rangle = \langle a \rangle, \\
b &:: \langle \rangle = \langle b \rangle, \\
\langle \rangle &:: \langle \rangle = \langle \langle \rangle \rangle.
\end{aligned}$$

Now if we take these three lists together with $\langle \rangle$, then with a and b we can construct many more lists. For example,

$$\begin{aligned}
a &:: \langle a \rangle = \langle a, a \rangle, \\
\langle a \rangle &:: \langle \rangle = \langle \langle a \rangle \rangle, \\
\langle \langle \rangle \rangle &:: \langle b \rangle = \langle \langle \langle \rangle \rangle, b \rangle, \\
\langle b \rangle &:: \langle \langle \rangle \rangle = \langle \langle b \rangle, \langle \rangle \rangle.
\end{aligned}$$

Using this idea, we'll make an inductive definition for the set S of all possible lists over $\{a, b\}$.

Basis: $\langle \rangle \in S$. (3.4)

Induction: If $x \in \{a, b\} \cup S$ and $L \in S$, then $x :: L \in S$.

end example

2	3	4	5	6
$\langle \rangle$	$\langle a \rangle$	$\langle \langle \rangle \rangle$	$\langle \langle a \rangle \rangle$	$\langle \langle \langle \rangle \rangle \rangle$
	$\langle b \rangle$	$\langle a, a \rangle$	$\langle \langle b \rangle \rangle$	$\langle \langle \rangle, \langle \rangle \rangle$
		$\langle a, b \rangle$	$\langle \langle \rangle, a \rangle$	$\langle a, a, \langle \rangle \rangle$
		$\langle b, a \rangle$	$\langle \langle \rangle, b \rangle$	$\langle a, \langle \rangle, a \rangle$
		$\langle b, b \rangle$	$\langle a, \langle \rangle \rangle$	$\langle \langle \rangle, a, a \rangle$
			$\langle b, \langle \rangle \rangle$	$\langle a, b, \langle \rangle \rangle$
			$\langle a, a, a \rangle$	$\langle a, b, a, b \rangle$
			$\vdots$	$\vdots$

Figure 3.1 A listing of lists by size.

3.1.4 Binary Trees

Recall that a binary tree is either empty or it has a root with a left and right subtree, each of which is a binary tree. This is an informal inductive description of the set of binary trees. To give a formal definition and to work with binary trees, we need some operations to pick off parts of a tree and to construct new trees.

In [Chapter 1](#) we represented binary trees by lists, where the empty binary tree is denoted by $\langle \rangle$ and a nonempty binary tree is denoted by the list L, x, R , where x is the root, L is the left subtree, and R is the right subtree. This gives us the ingredients for a more formal inductive definition of the set of all binary trees.

For convenience we'll let $\text{tree}(L, x, R)$ denote the binary tree with root x , left subtree L , and right subtree R . If we still want to represent binary trees as lists, then of course we can write

$$\text{tree}(L, x, R) = L, x, R.$$

Now suppose A is any set. Then we can describe the set B of all binary trees whose nodes come from A by saying that $\langle \rangle$ is in B , and if L and R are in B , then so is $\text{tree}(L, a, R)$ for any a in A . This gives us an inductive definition, which we can state formally as follows.

All Binary Trees over A

(3.5)

Basis: $\langle \rangle \in B$.

Induction: If $x \in A$ and $L, R \in B$, then $\text{tree}(L, x, R) \in B$.

We also have destructor operations for binary trees. We'll let *left*, *root*, and *right* denote the operations that return the left subtree, the root, and the right subtree, respectively, of a nonempty tree. For example, if

$$T = \text{tree}(L, x, R), \text{ then } \text{left}(T) = L, \text{ root}(T) = x, \text{ and } \text{right}(T) = R.$$

So for any nonempty binary tree T we have

$$T = \text{tree}(\text{left}(T), \text{root}(T), \text{right}(T)).$$

example 3.11 Binary Trees of Twins

Let $A = \{0, 1\}$. Suppose we need to work with the set *Twins* of all binary trees T over A that have the following property: The left and right subtrees of each node in T are identical in structure and node content. For example, *Twins* contains the empty tree and any single-node tree. *Twins* also contains the two trees shown in Figure 3.2.

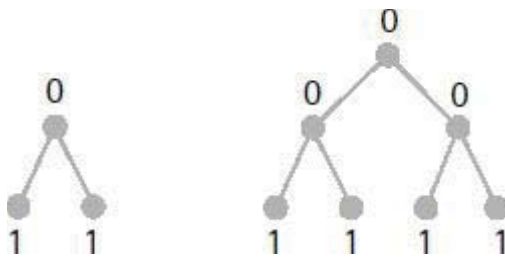


Figure 3.2 Twins as subtrees.

We can give an inductive definition of *Twins* by simply making sure that each new tree has the same left and right subtrees. Here's the definition:

Basis: $\langle \rangle \in \text{Twins}$.

Induction: If $x \in A$ and $T \in \text{Twins}$, then $\text{tree}(T, x, T) \in \text{Twins}$.

end example

example 3.12 Binary Trees of Opposites

Let $A = \{0, 1\}$, and suppose that *Opps* is the set of all nonempty binary trees T over A with the following property: The left and right subtrees of each node of T have identical structures, but the 0's and 1's are interchanged. For example, each single node tree is in *Opps*, as well as the two trees shown in Figure 3.3.

Since our set does not include the empty tree, the two singleton trees with nodes 1 and 0 should be the basis trees in *Opps*. The inductive definition of *Opps* can be given as follows:

Basis: $\text{tree}(\langle \rangle, 0, \langle \rangle), \text{tree}(\langle \rangle, 1, \langle \rangle) \in \text{Opps}$.

Induction: Let $x \in A$ and $T \in \text{Opps}$.

If $\text{root}(T) = 0$, then

$\text{tree}(T, x, \text{tree}(\text{right}(T), 1, \text{left}(T))) \in \text{Opps}$.

Otherwise,

$\text{tree}(T, x, \text{tree}(\text{right}(T), 0, \text{left}(T))) \in \text{Opps}$.

Does this definition work? Try out some examples. See whether the definition builds the four possible three-node trees.

end example

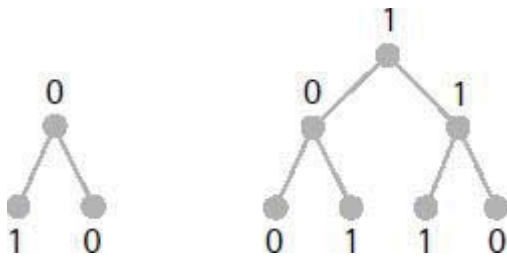


Figure 3.3 Opposites as subtrees.

3.1.5 Cartesian Products of Sets

Let's consider the problem of finding inductive definitions for subsets of the Cartesian product of two sets. For example, the set $\mathbb{N} \times \mathbb{N}$ can be defined inductively by starting with the pair $(0, 0)$ as the basis element. Then, for any pair (x, y) in the set, we can construct the following two pairs.

$$(x, y + 1) \text{ and } (x + 1, y).$$

It seems clear that this definition will define all elements of $\mathbb{N} \times \mathbb{N}$, although some pairs will be defined more than once. For example, the graph in Figure 3.4 shows four points. Notice that the point for the pair $(x + 1, y + 1)$ is

constructed from $(x, y + 1)$, but it is also constructed from $(x + 1, y)$.

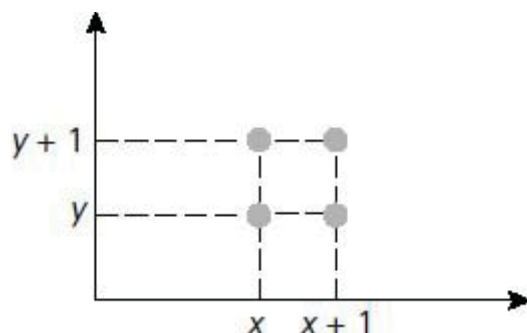


Figure 3.4 Four integer points.

example 3.13 Cartesian Product

A Cartesian product can be defined inductively if at least one set of the product can be defined inductively. For example, if A is any set, then we have the following inductive definition of $\mathbb{N} \times A$:

Basis: $(0, a) \in \mathbb{N} \times A$ for all $a \in A$.

Induction: If $(x, y) \in \mathbb{N} \times A$, then $(x + 1, y) \in \mathbb{N} \times A$.

end example

example 3.14 Part of a Plane

Let $S = \{(x, y) \mid x, y \in \mathbb{N} \text{ and } x \leq y\}$. From the point of view of a plane, S is the set of points in the first quadrant with integer coordinates on or above the main diagonal. We can define S inductively as follows:

Basis: $(0, 0) \in S$.

Induction: If $(x, y) \in S$, then $(x, y + 1), (x + 1, y + 1) \in S$.

For example, we can use $(0, 0)$ to construct $(0, 1)$ and $(1, 1)$. From $(0, 1)$ we construct $(0, 2)$ and $(1, 2)$. From $(1, 1)$ we construct $(1, 2)$ and $(2, 2)$. So some pairs get defined more than once.

end example

example 3.15 Describing an Area

Suppose we need to describe some area as a set of points. From a computational point of view, the area will be represented by discrete points, like pixels on a computer screen. So we can think of the area as a set of ordered pairs (x, y) forming a subset of $\mathbb{N} \times \mathbb{N}$.

To keep things simple we'll describe the area A under the curve of a function f between two points a and b on the x -axis. Figure 3.5 shows a general picture of the area A .

So the area A can be described as the following set of points, where $a, b \in \mathbb{N}$, and $f: \mathbb{N} \rightarrow \mathbb{N}$.

$$A = \{(x, y) \mid x, y \in \mathbb{N}, a \leq x \leq b, \text{ and } 0 \leq y \leq f(x)\}.$$

There are several ways we might proceed to give an inductive definition of A . For example, we can start with the point $(a, 0)$ on the x -axis. From $(a, 0)$ we can construct the column of points above it and the point $(a + 1, 0)$, from which the next column of points can be constructed. Here's the definition.

Basis: $(a, 0) \in A$.

Induction: If $(x, 0) \in A$ and $x < b$, then $(x + 1, 0) \in A$.

If $(x, y) \in A$ and $y < f(x)$, then $(x, y + 1) \in A$.

For example, the column of points $(a, 0), (a, 1), (a, 2), \dots, (a, f(a))$ is constructed by starting with the basis point $(a, 0)$ and by repeatedly using the second if-then statement. The first if-then statement constructs the points on the x -axis that are then used to construct the other columns of points. Notice with this definition that each pair is constructed exactly once.

end example

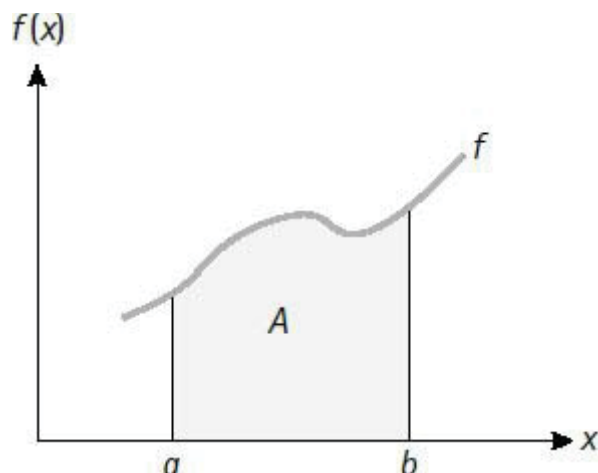


Figure 3.5 Area under a curve.

Exercises

Numbers

- For each of the following inductive definitions, start with the basis element and construct ten elements in the set.

a. *Basis:* $3 \in S$.

Induction: If $x \in S$, then $2x - 1 \in S$.

b. *Basis:* $1 \in S$.

Induction: If $x \in S$, then $2x, 2x + 1 \in S$.

- Find an inductive definition for each set S .

a. $\{1, 3, 5, 7, \dots\}$.

b. $\{0, 2, 4, 6, 8, \dots\}$.

c. $\{-3, -1, 1, 3, 5, \dots\}$.

d. $\{\dots, -7, -4, -1, 2, 5, 8, \dots\}$.

e. $\{1, 4, 9, 16, 25, \dots\}$.

f. $\{1, 3, 7, 15, 31, 63, \dots\}$.

- Find an inductive definition for each set S .

a. $\{4, 7, 10, 13, \dots\} \cup \{3, 6, 9, 12, \dots\}$.

- b. $\{3, 4, 5, 8, 9, 12, 16, 17, \dots\}$. *Hint:* Write the set as a union.
4. Find an inductive definition for each set S .
- $\{x \in \mathbb{N} \mid \text{floor}(x/2) \text{ is even}\}$.
 - $\{x \in \mathbb{N} \mid \text{floor}(x/2) \text{ is odd}\}$.
 - $\{x \in \mathbb{N} \mid x \bmod 5 = 2\}$.
 - $\{x \in \mathbb{N} \mid 2x \bmod 7 = 3\}$.
5. The following inductive definition was given in Example 3.4, the second robot example.

Basis: $\emptyset \in \text{Nat}$.

Induction: If $s \in \text{Nat}$, then $s \cup \{s\} \in \text{Nat}$.

In Example 3.4 we identified natural numbers with the elements of Nat by setting $0 = \emptyset$ and $n = n \cup \{n\}$ for $n \neq 0$. Show that $4 = \{0, 1, 2, 3\}$.

Strings

6. Find an inductive definition for each set S of strings.
- $\{a^n b c^n \mid n \in \mathbb{N}\}$.
 - $\{a^{2n} \mid n \in \mathbb{N}\}$.
 - $\{a^{2n+1} \mid n \in \mathbb{N}\}$.
 - $\{a^m b^n \mid m, n \in \mathbb{N}\}$.
 - $\{a^m b c^n \mid m, n \in \mathbb{N}\}$.
 - $\{a^m b^n \mid m, n \in \mathbb{N}, \text{ where } m > 0\}$.
 - $\{a^m b^n \mid m, n \in \mathbb{N}, \text{ where } n > 0\}$.
 - $\{a^m b^n \mid m, n \in \mathbb{N}, \text{ where } m > 0 \text{ and } n > 0\}$.
 - $\{a^m b^n \mid m, n \in \mathbb{N}, \text{ where } m > 0 \text{ or } n > 0\}$.
 - $\{a^{2n} \mid n \in \mathbb{N}\} \cup \{b^{2n+1} \mid n \in \mathbb{N}\}$.
 - $\{s \in \{a, b\}^* \mid s \text{ has the same number of } a\text{'s and } b\text{'s}\}$.
7. Find an inductive definition for each set S of strings. Note that a *palindrome* is a string that is the same when written in reverse order.

- a. Even palindromes over the set $\{a, b\}$.
 - b. Odd palindromes over the set $\{a, b\}$.
 - c. All palindromes over the set $\{a, b\}$.
 - d. The binary numerals.
8. Let the letters a , b , and c be constants; let the letters x , y , and z be variables; and let the letters f and g be functions of arity 1. We can define the set of terms over these symbols by saying that any constant or variable is a term and if t is a term, then so are $f(t)$ and $g(t)$. Find an inductive definition for the set T of terms.

Lists

9. For each of the following inductive definitions, start with the basis element and construct five elements in the set.
- a. *Basis:* $\langle a \rangle \in S$.
Induction: If $x \in S$, then $b :: x \in S$.
 - b. *Basis:* $\langle 1 \rangle \in S$.
Induction: If $x \in S$, then $2 \cdot \text{head}(x) :: x \in S$.
10. Find an inductive definition for each set S of lists. Use the cons constructor.
- a. $\{\langle a, a, a, a, a, a, \dots \rangle\}$.
 - b. $\{\langle 1, 2, 1, 3, 2, 1, \dots \rangle\}$.
 - c. $\{\langle a, b, b, a, a, a, b, b, b, a, a, a, b, b, b, a, \dots \rangle\}$.
 - d. $\{L \mid L \text{ has even length over } \{a\}\}$.
 - e. $\{L \mid L \text{ has even length over } \{0, 1, 2\}\}$.
 - f. $\{L \mid L \text{ has even length over a set } A\}$.
 - g. $\{L \mid L \text{ has odd length over } \{a\}\}$.
 - h. $\{L \mid L \text{ has odd length over } \{0, 1, 2\}\}$.
 - i. $\{L \mid L \text{ has odd length over a set } A\}$.
11. Find an inductive definition for each set S of lists. You may use the “consR” operation, where $\text{consR}(L, x)$ is the list constructed from the list L by adding a new element x on the right end. Similarly, you may use the

“headR” and “tailR” operations, which are like head and tail but look at things from the right side of a list.

- a. $\{\langle a \rangle, \langle a, b \rangle, \langle a, b, b \rangle, \dots\}$.
 - b. $\{\langle 1 \rangle, \langle 1, 2 \rangle, \langle 1, 2, 3 \rangle, \dots\}$.
 - c. $\{L \in \text{lists}(\{a, b\}) \mid L \text{ has the same number of } a\text{'s and } b\text{'s}\}$.
12. Find an inductive definition for the set S of all lists over $A = \{a, b\}$ that alternate a 's and b 's. For example, the lists $\langle \rangle$, $\langle a \rangle$, $\langle b \rangle$, $\langle a, b, a \rangle$, and $\langle b, a \rangle$ are in S . But $\langle a, a \rangle$ is not in S .

Binary Trees

13. Given the following inductive definition for a set S of binary trees. Start with the basis element and draw pictures of four binary trees in the set. Don't draw the empty subtrees.

Basis: $\text{tree}(\langle \rangle, a, \langle \rangle) \in S$.

Induction: If $T \in S$, then $\text{tree}(\text{tree}(\langle \rangle, a, \langle \rangle), a, T) \in S$.

14. Find an inductive definition for the set B of binary trees that represent arithmetic expressions that are either numbers in $\mathbb{N}$ or expressions that use operations $+$ or $-$.
15. Find an inductive definition for the set B of nonempty binary trees over $\{a\}$ in which each non-leaf node has two subtrees, one of which is a leaf and the other of which is either a leaf or a member of B .

Cartesian Products

16. Given the following inductive definition for a subset B of $\mathbb{N} \times \mathbb{N}$.

Basis: $(0, 0) \in B$.

Induction: If $(x, y) \in B$, then $(x + 1, y)$, $(x + 1, y + 1) \in B$.

- a. Describe the set B as a set of the form $\{(x, y) \mid \text{some property holds}\}$.
- b. Describe those elements in B that get defined in more than one way.

17. Find an inductive definition for each subset S of $\mathbb{N} \times \mathbb{N}$.

- a. $S = \{(x, y) \mid y = x \text{ or } y = x + 1\}$.
- b. $S = \{(x, y) \mid x \text{ is even and } y \leq x/2\}$.

18. Find an inductive definition for each product set S .

- a. $S = \text{lists}(A) \times \text{lists}(A)$ for some set A .
- b. $S = A \times \text{lists}(A)$.
- c. $S = \mathbb{N} \times \text{lists}(\mathbb{N})$.
- d. $S = \mathbb{N} \times \mathbb{N} \times \mathbb{N}$.

Proofs and Challenges

19. Let A be a set. Suppose O is the set of binary trees over A that contains an odd number of nodes. Similarly, let E be the set of binary trees over A that contains an even number of nodes. Find inductive definitions for O and E .
Hint: You can use O when defining E , and you can use E when defining O .

20. Use Example 3.15 as a guide to construct an inductive definition for the set of points in $\mathbb{N} \times \mathbb{N}$ that describe the area A between two curves f and g defined as follows for two natural numbers a and b :

$$A = \{(x, y) \mid x, y \in \mathbb{N}, a \leq x \leq b, \text{ and } g(x) \leq y \leq f(x)\}.$$

21. Prove that a set defined by (3.1) is countable if the basis elements in Step 1 are countable, the outside elements, if any, used in Step 2 are countable, and the rules specified in Step 2 are finite.

3.2 Recursive Functions and Procedures

Since we're going to be constructing functions and procedures in this section, we'd better agree on the idea of a procedure. From a computer science point of view a *procedure* is a program that performs one or more actions. So there is no requirement to return a specific value. For example, the execution of a statement like `print(x, y)` will cause the values of x and y to be printed. In this case, two actions are performed, and no values are returned. A procedure may also return one or more values through its argument list. For example, a

statement like $\text{allocate}(m, a, s)$ might perform the action of allocating a block of m memory cells and return the values a and s , where a is the beginning address of the block and the s tells whether the allocation was successful.

Definition of Recursively Defined

A function or a procedure is said to be *recursively defined* if it is defined in terms of itself. In other words, a function f is recursively defined if at least one value $f(x)$ is defined in terms of another value $f(y)$, where $x \neq y$. Similarly, a procedure P is recursively defined if the actions of P for some argument x are defined in terms of the actions of P for another argument y , where $x \neq y$.

Many useful recursively defined functions have domains that are inductively defined sets. Similarly, many recursively defined procedures process elements from inductively defined sets. For these cases there are very useful construction techniques. Let's describe the two techniques.

Constructing a Recursively Defined Function (3.6)

If S is an inductively defined set, then we can construct a function f with domain S as follows:

1. For each basis element $x \in S$, specify a value for $f(x)$.
2. Give rules that, for any inductively defined element $x \in S$, will define $f(x)$ in terms of previously defined values of f .

Any function constructed by (3.6) is recursively defined because it is defined in terms of itself by the induction part of the definition. In a similar way we can construct a recursively defined procedure to process the elements of an inductively defined set.

Constructing a Recursively Defined Procedure (3.7)

If S is an inductively defined set, we can construct a procedure P to process the elements of S as follows:

1. For each basis element $x \in S$, specify a set of actions for $P(x)$.
2. Give rules that, for any inductively defined element $x \in S$, will define the actions of $P(x)$ in terms of previously defined actions of P .

In the following paragraphs we'll see how (3.6) and (3.7) can be used to construct recursively defined functions and procedures over a variety of inductively defined sets. Most of our examples will be functions. But we'll define a few procedures too.

3.2.1 Numbers

Let's see how some number functions can be defined recursively. To illustrate the idea, suppose we want to calculate the sum of the first n natural numbers for any $n \in \mathbb{N}$. Letting $f(n)$ denote the desired sum, we can write the informal definition

$$f(n) = 0 + 1 + 2 + \dots + n.$$

We can observe, for example, that $f(0) = 0$, $f(1) = 1$, $f(2) = 3$, and so on. After a while we might notice that $f(3) = f(2) + 3 = 6$ and $f(4) = f(3) + 4 = 10$. In other words, when $n > 0$, the definition can be transformed in the following way:

$$\begin{aligned} f(n) &= 0 + 1 + 2 + \dots + n \\ &= (0 + 1 + 2 + \dots + (n-1)) + n \\ &= f(n-1) + n. \end{aligned}$$

This gives us the recursive part of a definition of f for any $n > 0$. For the basis case we have $f(0) = 0$. So we can write the following recursive definition for f :

$$\begin{aligned} f(0) &= 0, \\ f(n) &= (n-1) + n \quad \text{for } n > 0. \end{aligned}$$

There are two alternative forms that can be used to write a recursive

definition. One form expresses the definition as an *if-then-else* equation. For example, f can be described in the following way.

$$f(n) = \text{if } n = 0 \text{ then } 0 \text{ else } f(n - 1) + n.$$

Another form expresses the definition as equations whose left sides determine which equation to use in the evaluation of an expression rather than a conditional like $n > 0$. Such a form is called a *pattern-matching* definition because the equation chosen to evaluate $f(x)$ is determined uniquely by which left side $f(x)$ matches. For example, f can be described in the following way.

$$\begin{aligned} f(0) &= 0, \\ f(n + 1) &= f(n) + n + 1. \end{aligned}$$

For example, $f(3)$ matches $f(n + 1)$ with $n = 2$, so we would choose the second equation to evaluate $f(3) = f(2) + 3$, and so on.

A recursively defined function can be evaluated by a technique called *unfolding* the definition. For example, we'll evaluate the expression $f(4)$.

$$\begin{aligned} f(4) &= f(3) + 4 \\ &= f(2) + 3 + 4 \\ &= f(1) + 2 + 3 + 4 \\ &= f(0) + 1 + 2 + 3 + 4 \\ &= 0 + 1 + 2 + 3 + 4 \\ &= 10. \end{aligned}$$

example 3.16 Using the Floor Function

Let $f: \mathbb{N} \rightarrow \mathbb{N}$ be defined in terms of the floor function as follows:

$$\begin{aligned} f(0) &= 0, \\ f(n) &= f(\text{floor}(n/2)) + n \quad \text{for } n > 0. \end{aligned}$$

Notice in this case that $f(n)$ is not defined in terms of $f(n - 1)$ but rather in

terms of $f(\text{floor}(n/2))$. For example, $f(16) = f(8) + 16$. The first few values are $f(0) = 0, f(1) = 1, f(2) = 3, f(3) = 4$, and $f(4) = 7$. We'll calculate $f(25)$.

$$\begin{aligned}
 f(25) &= f(12) + 25 \\
 &= f(6) + 12 + 25 \\
 &= f(3) + 6 + 12 + 25 \\
 &= f(1) + 3 + 6 + 12 + 25 \\
 &= f(0) + 1 + 3 + 6 + 12 + 25 \\
 &= 0 + 1 + 3 + 6 + 12 + 25 \\
 &= 47.
 \end{aligned}$$

end example

example 3.17 Adding Odd Numbers

Let $f: \mathbb{N} \rightarrow \mathbb{N}$ denote the function to add up the first n odd natural numbers. So f has the following informal definition.

$$f(n) = 1 + 3 + \cdots + (2n + 1).$$

For example, the definition tells us that $f(0) = 1$. For $n > 0$ we can make the following transformation of $f(n)$ into an expression in terms of $f(n - 1)$:

$$\begin{aligned}
 f(n) &= 1 + 3 + \cdots + (2n + 1) \\
 &= (1 + 3 + \cdots + (2(n - 1) + 1)) + (2n + 1) \\
 &= f(n - 1) + 2n + 1.
 \end{aligned}$$

So we can make the following recursive definition of f :

$$\begin{aligned}
 f(0) &= 1, \\
 f(n) &= f(n - 1) + 2n + 1 \quad \text{for } n > 0.
 \end{aligned}$$

Alternatively, we can write the recursive part of the definition as

$$f(n + 1) = f(n) + 2n + 3.$$

We can also write the definition in the following if-then-else form.

$$f(n) = \text{if } n = 0 \text{ then } 1 \text{ else } f(n - 1) + 2n + 1.$$

Here is the evaluation of $f(3)$ using the if-then-else definition:

$$\begin{aligned} f(3) &= f(2) + 2(3) + 1 \\ &= f(1) + 2(2) + 1 + 2(3) + 1 \\ &= f(0) + 2(1) + 1 + 2(2) + 1 + 2(3) + 1 \\ &= 1 + 2(1) + 1 + 2(2) + 1 + 2(3) + 1 \\ &= 1 + 3 + 5 + 7 \\ &= 16. \end{aligned}$$

end example

example 3.18 The Rabbit Problem

The *Fibonacci numbers* are the numbers in the sequence

$$0, 1, 1, 2, 3, 5, 8, 13, \dots$$

where each number after the first two is computed by adding the preceding two numbers. These numbers are named after the mathematician Leonardo Fibonacci, who in 1202 introduced them in his book *Liber Abaci*, in which he proposed and solved the following problem: Starting with a pair of rabbits, how many pairs of rabbits can be produced from that pair in a year if it is assumed that every month each pair produces a new pair that becomes productive after one month?

For example, if we don't count the original pair and assume that the original pair needs one month to mature and that no rabbits die, then the number of new pairs produced each month for 12 consecutive months is given by the sequence

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89.

The sum of these numbers, which is 232, is the number of pairs of rabbits produced in one year from the original pair.

Fibonacci numbers seem to occur naturally in many unrelated problems. Of course, they can also be defined recursively. For example, letting $\text{fib}(n)$ be the n th Fibonacci number, we can define fib recursively as follows:

$$\begin{aligned}\text{fib}(0) &= 0, \\ \text{fib}(1) &= 1, \\ \text{fib}(n) &= \text{fib}(n-1) + \text{fib}(n-2) \quad \text{for } n \geq 2.\end{aligned}$$

The third line could be written in pattern-matching form as

$$\text{fib}(n+2) = \text{fib}(n+1) + \text{fib}(n).$$

The definition of fib in if-then-else form looks like

$$\begin{aligned}\text{fib}(n) &= \text{if } n = 0 \text{ then } 0 \\ &\quad \text{else if } n = 1 \text{ then } 1 \\ &\quad \text{else } \text{fib}(n-1) + \text{fib}(n-2).\end{aligned}$$

end example

3.2.2 Strings

Let's see how some string functions can be defined recursively. To illustrate the idea, suppose we want to calculate the complement of any string over the alphabet $\{a, b\}$. For example, the complement of the string $bbab$ is $aaba$.

Let $f(x)$ be the complement of x . To find a recursive definition for f we'll start by observing that an arbitrary string over $\{a, b\}$ is either Λ or has the form ay or by for some string y . So we'll define the result of f applied to each of these forms as follows:

$$\begin{aligned}
 f(\Lambda) &= \Lambda, \\
 f(ax) &= bf(x), \\
 f(bx) &= af(x).
 \end{aligned}$$

For example, we'll evaluate $f(bbab)$:

$$\begin{aligned}
 f(bbab) &= af(bab) \\
 &= aaf(ab) \\
 &= aabf(b) \\
 &= aabaf(\Lambda) \\
 &= abba\Lambda \\
 &= abba.
 \end{aligned}$$

Here are some more examples.

example 3.19 Prefixes of Strings

Consider the problem of finding the longest common prefix of two strings. A string p is a *prefix* of the string x if x can be written in the form $x = ps$ for some string s . For example, aab is the longest common prefix of the two strings $aabbab$ and $aababb$.

For two strings x and y over $\{a, b\}$, let $f(x, y)$ be the longest common prefix of x and y . To find a recursive definition for f we can start by observing that an arbitrary string over $\{a, b\}$ is either the empty string Λ or has the form as or bs for some string s . In other words, the strings over $\{a, b\}$ are an inductively defined set. Here is a definition of f in pattern-matching form:

$$\begin{aligned}
f(\Lambda, x) &= \Lambda, \\
f(x, \Lambda) &= \Lambda, \\
f(as, bt) &= \Lambda, \\
f(bs, at) &= \Lambda, \\
f(as, at) &= af(s, t), \\
f(bs, bt) &= bf(s, t).
\end{aligned}$$

We can put the definition in if-then-else form as follows:

$$\begin{aligned}
f(x, y) &= \text{if } x = \Lambda \text{ or } y = \Lambda \text{ then } \Lambda \\
&\quad \text{else if } x = as \text{ and } y = at \text{ then } af(s, t) \\
&\quad \text{else if } x = bs \text{ and } y = bt \text{ then } bf(s, t) \\
&\quad \text{else } \Lambda.
\end{aligned}$$

We'll demonstrate the definition of f by calculating $f(aabbab, aababb)$:

$$\begin{aligned}
f(aabbab, aababb) &= af(abbab, ababb) \\
&= aaf(bbab, babb) \\
&= aabf(bab, abb) \\
&= aab\Lambda \\
&= aab.
\end{aligned}$$

end example

example 3.20 Converting Natural Numbers to Binary

Recall from Section 2.1 that we can represent a natural number x as

$$x = 2(\text{floor}(x/2)) + x \bmod 2.$$

This formula can be used to create a binary representation of x because $x \bmod$

2 is the rightmost bit of the representation. The next bit is found by computing $\text{floor}(x/2) \bmod 2$. The next bit is $\text{floor}(\text{floor}(x/2)/2) \bmod 2$, and so on. For example, we'll compute the binary representation of 13.

$$\begin{array}{rclclcl} 13 & = & 2 \text{floor}(13/2) + 13 \bmod 2 & = & 2(6) + 1 \\ 6 & = & 2 \text{floor}(6/2) + 6 \bmod 2 & = & 2(3) + 0 \\ 3 & = & 2 \text{floor}(3/2) + 3 \bmod 2 & = & 2(1) + 1 \\ 1 & = & 2 \text{floor}(1/2) + 1 \bmod 2 & = & 2(0) + 1 \end{array}$$

We can read off the remainders in reverse order to obtain 1101, which is the binary representation of 13.

Let's try to use this idea to write a recursive definition for the function "binary" to compute the binary representation for a natural number. If $x = 0$ or $x = 1$, then x is its own binary representation. If $x > 1$, then the binary representation of x is that of $\text{floor}(x/2)$ with the bit $x \bmod 2$ attached on the right end. So our recursive definition of binary can be written as follows, where "cat" is the string concatenation function.

$$\begin{array}{ll} \text{binary}(0) = 0, & (3.8) \\ \text{binary}(1) = 1, \\ \text{binary}(x) = \text{cat}(\text{binary}(\text{floor}(x/2)), x \bmod 2) & \text{for } x > 1. \end{array}$$

The definition can be written in if-then-else form as

$$\begin{array}{l} \text{binary}(x) = \text{if } x = 0 \text{ or } x = 1 \text{ then } x \\ \quad \text{else cat}(\text{binary}(\text{floor}(x/2)), x \bmod 2). \end{array}$$

For example, we'll unfold the definition to calculate $\text{binary}(13)$:

$$\begin{aligned}
\text{binary}(13) &= \text{cat}(\text{binary}(6), 1) \\
&= \text{cat}(\text{cat}(\text{binary}(3), 0), 1) \\
&= \text{cat}(\text{cat}(\text{cat}(\text{binary}(1), 1), 0), 1) \\
&= \text{cat}(\text{cat}(\text{cat}(1, 1), 0), 1) \\
&= \text{cat}(\text{cat}(11, 0), 1) \\
&= \text{cat}(110, 1) \\
&= 1101.
\end{aligned}$$

end example

3.2.3 Lists

Let's see how some functions that use lists can be defined recursively. To illustrate the idea, suppose we need to define the function $f: \mathbb{N} \rightarrow \text{lists}(\mathbb{N})$ that computes the following backward sequence:

$$f(n) = \langle n, n-1, \dots, 1, 0 \rangle.$$

With a little help from the cons function for lists, we can transform the informal definition of $f(n)$ into a computable expression in terms of $f(n-1)$:

$$\begin{aligned}
f(n) &= \langle n, n-1, \dots, 1, 0 \rangle \\
&= \text{cons}(n, \langle n-1, \dots, 1, 0 \rangle) \\
&= \text{cons}(n, f(n-1)).
\end{aligned}$$

Therefore, f can be defined recursively by

$$\begin{aligned}
f(0) &= \langle 0 \rangle, \\
f(n) &= \text{cons}(n, f(n-1)) \text{ for } n > 0.
\end{aligned}$$

This definition can be written in if-then-else form as

$$f(n) = \text{if } n = 0 \text{ then } \langle 0 \rangle \text{ else } \text{cons}(n, f(n-1)).$$

To see how the evaluation works, look at the unfolding that results when we evaluate $f(3)$:

$$\begin{aligned}
 f(3) &= \text{cons}(3, f(2)) \\
 &= \text{cons}(3, \text{cons}(2, f(1))) \\
 &= \text{cons}(3, \text{cons}(2, \text{cons}(1, f(0)))) \\
 &= \text{cons}(3, \text{cons}(2, \text{cons}(1, \langle 0 \rangle))) \\
 &= \text{cons}(3, \text{cons}(2, \langle 1, 0 \rangle)) \\
 &= \text{cons}(3, \langle 2, 1, 0 \rangle) \\
 &= \langle 3, 2, 1, 0 \rangle.
 \end{aligned}$$

We haven't given a recursively defined procedure yet. So let's give one for the problem we've been discussing. For example, suppose that $P(n)$ prints out the numbers in the list $\langle n, n - 1, \dots, 0 \rangle$. A recursive definition of P can be written as follows.

```

P(n):  if n = 0 then print(0)
        else
            print(n);
            P(n - 1)
        fi.

```

example 3.21 Length of a List

Let S be a set and let “length” be the function of type $\text{lists}(S) \rightarrow \mathbb{N}$, which returns the number of elements in a list. We can define length recursively by noticing that the length of an empty list is zero and the length of a nonempty list is one plus the length of its tail. A definition follows.

$$\begin{aligned}
 \text{length}(\langle \rangle) &= 0, \\
 \text{length}(\text{cons}(x, t)) &= 1 + \text{length}(t).
 \end{aligned}$$

Recall that the infix form of $\text{cons}(x, t)$ is $x :: t$. So we could just as well write the second equation as

$$\text{length}(x :: t) = 1 + \text{length}(t).$$

Also, we could write the recursive part of the definition with a condition as follows:

$$\text{length}(L) = 1 + \text{length}(\text{tail}(L)) \quad \text{for } L \neq \langle \rangle.$$

In if-then-else form the definition can be written as follows:

$$\text{length}(L) = \text{if } L = \langle \rangle \text{ then } 0 \text{ else } 1 + \text{length}(\text{tail}(L)).$$

The length function can be evaluated by unfolding its definition. For example, we'll evaluate $\text{length}(a, b, c)$.

$$\begin{aligned} \text{length}(\langle a, b, c \rangle) &= 1 + \text{length}(\langle b, c \rangle) \\ &= 1 + 1 + \text{length}(\langle c \rangle) \\ &= 1 + 1 + 1 + \text{length}(\langle \rangle) \\ &= 1 + 1 + 1 + 0 \\ &= 3. \end{aligned}$$

end example

example 3.22 The Distribute Function

Suppose we want to write a recursive definition for the distribute function, which we'll denote by "dist." Recall, for example, that

$$\text{dist}(a, b, c, d, e) = (a, b), (a, c), (a, d), (a, e).$$

To discover the recursive part of the definition, we'll rewrite the example equation by splitting up the lists into head and tail components as follows:

$$\begin{aligned}
\text{dist}(a, \langle b, c, d, e \rangle) &= \langle (a, b), (a, c), (a, d), (a, e) \rangle \\
&= (a, b) :: \langle (a, c), (a, d), (a, e) \rangle \\
&= (a, b) :: \text{dist}(a, \langle c, d, e \rangle).
\end{aligned}$$

That's the key to the recursive part of the definition. Since we are working with lists, the basis case is $\text{dist}(a, \langle \rangle)$, which we define as $\langle \rangle$. So the recursive definition can be written as follows:

$$\begin{aligned}
\text{dist}(x, \langle \rangle) &= \langle \rangle, \\
\text{dist}(x, h :: T) &= (x, h) :: \text{dist}(x, T).
\end{aligned}$$

For example, we'll evaluate the expression $\text{dist}(3, \langle 10, 20 \rangle)$:

$$\begin{aligned}
\text{dist}(3, \langle 10, 20 \rangle) &= (3, 10) :: \text{dist}(3, \langle 20 \rangle) \\
&= (3, 10) :: (3, 20) :: \text{dist}(3, \langle \rangle) \\
&= (3, 10) :: (3, 20) :: \langle \rangle \\
&= (3, 10) :: \langle (3, 20) \rangle \\
&= \langle (3, 10), (3, 20) \rangle.
\end{aligned}$$

An if-then-else definition of dist takes the following form:

$$\begin{aligned}
\text{dist}(x, L) &= \text{if } L = \langle \rangle \text{ then } \langle \rangle \\
&\quad \text{else } (x, \text{head}(L)) :: \text{dist}(x, \text{tail}(L)).
\end{aligned}$$

end example

example 3.23 The Pairs Function

Recall that the “pairs” function creates a list of pairs of corresponding elements from two lists. For example,

$$\text{pairs}(\langle a, b, c \rangle, \langle 1, 2, 3 \rangle) = \langle (a, 1), (b, 2), (c, 3) \rangle.$$

To discover the recursive part of the definition, we'll rewrite the example equation by splitting up the lists into head and tail components as follows:

$$\begin{aligned}\text{pairs}(\langle a, b, c \rangle, \langle 1, 2, 3 \rangle) &= \langle (a, 1), (b, 2), (c, 3) \rangle \\ &= (a, 1) :: \langle (b, 2), (c, 3) \rangle \\ &= (a, 1) :: \text{pairs}(\langle b, c \rangle, \langle 2, 3 \rangle).\end{aligned}$$

Now the pairs function can be defined recursively by the following equations:

$$\begin{aligned}\text{pairs}(\langle \rangle, \langle \rangle) &= \langle \rangle, \\ \text{pairs}(x :: T, y :: U) &= (x, y) :: \text{pairs}(T, U).\end{aligned}$$

For example, we'll evaluate the expression $\text{pairs}(\langle a, b \rangle, \langle 1, 2 \rangle)$:

$$\begin{aligned}\text{pairs}(\langle a, b \rangle, \langle 1, 2 \rangle) &= (a, 1) :: \text{pairs}(\langle b \rangle, \langle 2 \rangle) \\ &= (a, 1) :: (b, 2) :: \text{pairs}(\langle \rangle, \langle \rangle) \\ &= (a, 1) :: (b, 2) :: \langle \rangle \\ &= (a, 1) :: \langle (b, 2) \rangle \\ &= \langle (a, 1), (b, 2) \rangle.\end{aligned}$$

end example

example 3.24 The ConsRight Function

Suppose we need to give a recursive definition for the sequence function. Recall, for example, that $\text{seq}(4) = 0, 1, 2, 3, 4$. Good old “cons” doesn't seem up to the task. For example, if we somehow have computed $\text{seq}(3)$, then $\text{cons}(4, \text{seq}(3)) = 4, 0, 1, 2, 3$. It would be nice if we had a constructor to place an element on the right of a list, just as cons places an element on the left of a list. We'll write a definition for the function “consR” to do just that. For example, we want

$$\text{consR}(a, b, c, d) = a, b, c, d.$$

We can get an idea of how to proceed by rewriting the previous equation as follows in terms of the infix form of cons:

$$\begin{aligned}\text{consR}(\langle a, b, c \rangle, d) &= \langle a, b, c, d \rangle \\ &= a :: \langle b, c, d \rangle \\ &= a :: \text{consR}(\langle b, c \rangle, d).\end{aligned}$$

So the clue is to split the list $\langle a, b, c \rangle$ into its head and tail. We can write the recursive definition of consR using the if-then-else form as follows:

$$\begin{aligned}\text{consR}(L, a) &= \text{if } L = \langle \rangle \text{ then } \langle a \rangle \\ &\quad \text{else head}(L) :: \text{consR}(\text{tail}(L), a).\end{aligned}$$

This definition can be written in pattern-matching form as follows:

$$\begin{aligned}\text{consR}(\langle \rangle, a) &= a :: \langle \rangle, \\ \text{consR}(b :: T, a) &= b :: \text{consR}(T, a).\end{aligned}$$

For example, we can construct the list $\langle x, y \rangle$ with consR as follows:

$$\begin{aligned}\text{consR}(\text{consR}(\langle \rangle, x), y) &= \text{consR}(x :: \langle \rangle, y) \\ &= x :: \text{consR}(\langle \rangle, y) \\ &= x :: y :: \langle \rangle \\ &= x :: \langle y \rangle \\ &= \langle x, y \rangle.\end{aligned}$$

end example

example 3.25 Concatenation of Lists

An important operation on lists is the concatenation of two lists into a single list. Let “cat” denote the concatenation function. Its type is $\text{lists}(A) \times \text{lists}(A)$

→ lists(A). For example,

$$\text{cat}(\langle a, b \rangle, \langle c, d \rangle) = \langle a, b, c, d \rangle.$$

Since both arguments are lists, we have some choices to make. Notice, for example, that we can rewrite the equation as follows:

$$\begin{aligned}\text{cat}(\langle a, b \rangle, \langle c, d \rangle) &= \langle a, b, c, d \rangle \\ &= a :: \langle b, c, d \rangle \\ &= a :: \text{cat}(\langle b \rangle, \langle c, d \rangle).\end{aligned}$$

So the recursive part can be written in terms of the head and tail of the first argument list. Here's an if-then-else definition for cat.

$$\begin{aligned}\text{cat}(L, M) &= \text{if } L = \langle \rangle \text{ then } M \\ &\quad \text{else head}(L) :: \text{cat}(\text{tail}(L), M).\end{aligned}$$

We'll unfold the definition to evaluate the expression $\text{cat}(\langle a, b \rangle, \langle c, d \rangle)$:

$$\begin{aligned}\text{cat}(\langle a, b \rangle, \langle c, d \rangle) &= a :: \text{cat}(\langle b \rangle, \langle c, d \rangle) \\ &= a :: b :: \text{cat}(\langle \rangle, \langle c, d \rangle) \\ &= a :: b :: \langle c, d \rangle \\ &= a :: \langle b, c, d \rangle \\ &= \langle a, b, c, d \rangle.\end{aligned}$$

end example

example 3.26 Sorting a List by Insertion

Let's define a function to sort a list of numbers by repeatedly inserting a new number into an already sorted list of numbers. Suppose "insert" is a function that does this job. Then the sort function itself is easy. For a basis case, notice that the empty list is already sorted. For the recursive case we sort the list $x ::$

L by inserting x into the list obtained by sorting L . The definition can be written as follows:

$$\begin{aligned}\text{sort}(\langle \rangle) &= \langle \rangle, \\ \text{sort}(x :: L) &= \text{insert}(x, \text{sort}(L)).\end{aligned}$$

Everything seems to make sense as long as `insert` does its job. We'll assume that whenever the number to be inserted is already in the list, then a new copy will be placed to the left of the one already there. Now let's define `insert`. Again, the basis case is easy. The empty list is sorted, and to insert x into $\langle \rangle$, we simply create the singleton list x . Otherwise—if the sorted list is not empty—either x belongs on the left of the list, or it should actually be inserted somewhere else in the list. An if-then-else definition can be written as follows:

$$\begin{aligned}\text{insert}(x, S) &= \text{if } S = \langle \rangle \text{ then } \langle x \rangle \\ &\quad \text{else if } x \leq \text{head}(S) \text{ then } x :: S \\ &\quad \text{else } \text{head}(S) :: \text{insert}(x, \text{tail}(S)).\end{aligned}$$

Notice that `insert` works only when S is already sorted. For example, we'll unfold the definition of `insert(3, 1, 2, 6, 8)`:

$$\begin{aligned}\text{insert}(3, \langle 1, 2, 6, 8 \rangle) &= 1 :: \text{insert}(3, \langle 2, 6, 8 \rangle) \\ &= 1 :: 2 :: \text{insert}(3, \langle 6, 8 \rangle) \\ &= 1 :: 2 :: 3 :: \langle 6, 8 \rangle \\ &= \langle 1, 2, 3, 6, 8 \rangle.\end{aligned}$$

end example

example 3.27 The Map Function

We'll construct a recursive definition for the `map` function. Recall, for example that

$$\text{map}(f, \langle a, b, c \rangle) = \langle f(a), f(b), f(c) \rangle.$$

Since the second argument is a list, it makes sense to define the basis case as $\text{map}(f, \langle \rangle) = \langle \rangle$. To discover the recursive part of the definition, we'll rewrite the example equation as follows:

$$\begin{aligned} \text{map}(f, \langle a, b, c \rangle) &= \langle f(a), f(b), f(c) \rangle \\ &= f(a) :: \langle f(b), f(c) \rangle \\ &= f(a) :: \text{map}(f, \langle b, c \rangle). \end{aligned}$$

So the recursive part can be written in terms of the head and tail of the input list. Here's an if-then-else definition for map.

$$\begin{aligned} \text{map}(f, L) &= \text{if } L = \langle \rangle \text{ then } \langle \rangle \\ &\quad \text{else } f(\text{head}(L)) :: \text{map}(f, \text{tail}(L)). \end{aligned}$$

For example, we'll evaluate the expression $\text{map}(f, \langle a, b, c \rangle)$.

$$\begin{aligned} \text{map}(f, \langle a, b, c \rangle) &= f(a) :: \text{map}(f, \langle b, c \rangle) \\ &= f(a) :: f(b) :: \text{map}(f, \langle c \rangle) \\ &= f(a) :: f(b) :: f(c) :: \text{map}(f, \langle \rangle) \\ &= f(a) :: f(b) :: f(c) :: \langle \rangle \\ &= \langle f(a), f(b), f(c) \rangle. \end{aligned}$$

end example

3.2.4 Binary Trees

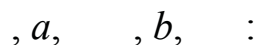
Let's look at some functions that compute properties of binary trees. To start, suppose we need to know the number of nodes in a binary tree. Since the set of binary trees over a particular set can be defined inductively, we should be able to come up with a recursively defined function that suits our needs. Let "nodes" be the function that returns the number of nodes in a binary tree. Since

the empty tree has no nodes, we have $\text{nodes}(\langle \rangle) = 0$. If the tree is not empty, then the number of nodes can be computed by adding 1 to the number of nodes in the left and right subtrees. The definition of nodes can be written as follows:

$$\begin{aligned}\text{nodes}(\langle \rangle) &= 0, \\ \text{nodes}(\text{tree}(L, a, R)) &= 1 + \text{nodes}(L) + \text{nodes}(R).\end{aligned}$$

If we want the corresponding if-then-else form of the definition, it looks like

$$\begin{aligned}\text{nodes}(T) &= \text{if } T = \langle \rangle \text{ then } 0 \\ &\quad \text{else } 1 + \text{nodes}(\text{left}(T)) + \text{nodes}(\text{right}(T)).\end{aligned}$$

For example, we'll evaluate $\text{nodes}(T)$ for $T =$  :

$$\begin{aligned}\text{nodes}(T) &= 1 + \text{nodes}(\langle \langle \rangle, a, \langle \rangle \rangle) + \text{nodes}(\langle \rangle) \\ &= 1 + 1 + \text{nodes}(\langle \rangle) + \text{nodes}(\langle \rangle) + \text{nodes}(\langle \rangle) \\ &= 1 + 1 + 0 + 0 + 0 \\ &= 2.\end{aligned}$$

example 3.28 A Binary Search Tree

Suppose we have a binary search tree whose nodes are numbers, and we want to add a new number to the tree, under the assumption that the new tree is still a binary search tree. A function to do the job needs two arguments, a number x and a binary search tree T . Let the name of the function be “insert.”

The basis case is easy. If $T =$ , then return $\text{tree}(\text{ }, x, \text{ })$. The recursive part is straightforward. If $x < \text{root}(T)$, then we need to replace the subtree $\text{left}(T)$ by $\text{insert}(x, \text{left}(T))$. Otherwise, we replace $\text{right}(T)$ by $\text{insert}(x, \text{right}(T))$. Notice that repeated elements are entered to the right. If we didn't want to add repeated elements, then we could simply return T whenever $x = \text{root}(T)$. The if-then-else form of the definition is

```

insert (x, T) = if T = ⟨ ⟩ then tree (⟨ ⟩, x, ⟨ ⟩)
               else if x < root (T) then
                   tree (insert (x, left (T)), root (T), right (T))
               else
                   tree (left (T), root (T), insert (x, right (T))) .

```

Now suppose we want to build a binary search tree from a given list of numbers in which the numbers are in no particular order. We can use the insert function as the main ingredient in a recursive definition. Let “makeTree” be the name of the function. We’ll use two variables to describe the function, a binary search tree T and a list of numbers L .

```

makeTree (T, L) = if L = ⟨ ⟩ then T
                  else makeTree (insert (head (L), T), tail (L)) .

```

(3.9)

To construct a binary search tree with this function, we apply makeTree to the pair of arguments $(\langle \rangle, L)$. As an example, the reader should unfold the definition for makeTree($\langle \rangle, 3, 2, 4$).

The function makeTree can be defined another way. Suppose we consider the following definition for constructing a binary search tree:

```

makeTree (T, L) = if L = ⟨ ⟩ then T
                  else insert (head (L), makeTree (T, tail (L))) .

```

(3.10)

You should evaluate the expression makeTree($\langle \rangle, \langle 3, 2, 4 \rangle$) by unfolding this alternative definition. It should help explain the difference between the two definitions.

end example

Traversing Binary Trees

There are several useful ways to list the nodes of a binary tree. The three most popular methods of traversing a binary tree are called *preorder*, *inorder*, and *postorder*. We’ll start with the definition of a preorder traversal.

Preorder Traversal

The *preorder* traversal of a binary tree starts by visiting the root. Then there is a preorder traversal of the left subtree followed by a preorder traversal of the right subtree.

For example, the preorder listing of the nodes of the binary tree in [Figure 3.6](#) is $\langle a, b, c, d, e \rangle$. It's common practice to write the listing without any punctuation symbols as

$a\ b\ c\ d\ e.$

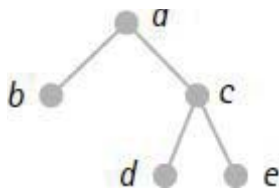


Figure 3.6 A binary tree.

example 3.29 A Preorder Procedure

Since binary trees are inductively defined, we can easily write a recursively defined procedure to output the preorder listing of a binary tree. For example, the following recursively defined procedure prints the preorder listing of its argument T .

```
Preorder( $T$ ):  if  $T \neq \langle \rangle$  then  
               print(root( $T$ ));  
               Preorder(left( $T$ ));  
               Preorder(right( $T$ ))  
            fi.
```

end example

example 3.30 A Preorder Function

Now let's write a function to compute the preorder listing of a binary tree. Letting "preOrd" be the name of the preorder function, a definition can be written as follows:

$$\begin{aligned}\text{preOrd}(\langle \rangle) &= \langle \rangle, \\ \text{preOrd}(\text{tree}(L, x, R)) &= x :: \text{cat}(\text{preOrd}(L), \text{preOrd}(R)).\end{aligned}$$

The if-then-else form of preOrd can be written as follows:

$$\begin{aligned}\text{preOrd}(T) &= \text{if } T = \langle \rangle \text{ then } \langle \rangle \\ &\quad \text{else } \text{root}(T) :: \text{cat}(\text{preOrd}(\text{left}(T)), \text{preOrd}(\text{right}(T))).\end{aligned}$$

We'll evaluate the expression preOrd(T) for the tree $T = \langle \langle \langle \rangle, a, \langle \rangle \rangle, b, \langle \rangle \rangle$:

$$\begin{aligned}\text{preOrd}(T) &= b :: \text{cat}(\text{preOrd}(\langle \langle \rangle, a, \langle \rangle \rangle), \text{preOrd}(\langle \rangle)) \\ &= b :: \text{cat}(a :: \text{cat}(\text{preOrd}(\langle \rangle), \text{preOrd}(\langle \rangle)), \text{preOrd}(\langle \rangle)) \\ &= b :: \text{cat}(a :: \langle \rangle, \langle \rangle) \\ &= b :: \text{cat}(\langle a \rangle, \langle \rangle) \\ &= b :: \langle a \rangle \\ &= \langle b, a \rangle.\end{aligned}$$

end example

The definitions for the inorder and postorder traversals of a binary tree are similar to the preorder traversal. The only difference is when the root is visited during the traversal.

Inorder Traversal

The *inorder* traversal of a binary tree starts with an inorder traversal of the left subtree. Then the root is visited. Lastly, there is an inorder traversal of the right subtree.

For example, the inorder listing of the tree in [Figure 3.6](#) is

$b a d c e.$

Postorder Traversal

The *postorder* traversal of a binary tree starts with a postorder traversal of the left subtree and is followed by a postorder traversal of the right subtree. Lastly, the root is visited.

The postorder listing of the tree in [Figure 3.6](#) is

$b d e c a.$

We'll leave the construction of the inorder and postorder procedures and functions as exercises.

3.2.5 Two More Problems

We'll look at two more problems, each of which requires a little extra thinking on the way to a solution.

The Repeated Element Problem

Suppose we want to remove repeated elements from a list. Depending on how we proceed, there might be different solutions. For example, we can remove the repeated elements from the list $\langle u, g, u, h, u \rangle$ in three ways, depending on which occurrence of u we keep: $\langle u, g, h \rangle$, $\langle g, u, h \rangle$, or $\langle g, h, u \rangle$. We'll solve the problem by always keeping the leftmost occurrence of each element. Let "remove" be the function that takes a list L and returns the list $\text{remove}(L)$, which has no repeated elements and contains the leftmost occurrence of each element of L .

To start things off, we can say $\text{remove}(\langle \rangle) = \langle \rangle$. Now if $L \neq \langle \rangle$, then L has the form $L = b :: M$ for some list M . In this case, the head of $\text{remove}(L)$ should be b . The tail of $\text{remove}(L)$ can be obtained by removing all occurrences of b from M and then removing all repeated elements from the resulting list. So we

need a new function to remove all occurrences of an element from a list.

Let $\text{removeAll}(b, M)$ denote the list obtained from M by removing all occurrences of b . Now we can write a definition for the remove function as follows:

$$\begin{aligned}\text{remove}(\langle \rangle) &= \langle \rangle, \\ \text{remove}(b :: M) &= b :: \text{remove}(\text{removeAll}(b, M)).\end{aligned}$$

We can rewrite the solution in if-then-else form as follows:

$$\begin{aligned}\text{remove}(L) &= \text{if } L = \langle \rangle \text{ then } \langle \rangle \\ &\quad \text{else head}(L) :: \text{remove}(\text{removeAll}(\text{head}(L), \text{tail}(L))).\end{aligned}$$

To complete the task, we need to define the “ removeAll ” function. The basis case is $\text{removeAll}(b, \langle \rangle) = \langle \rangle$. If $M \neq \langle \rangle$, then the value of $\text{removeAll}(b, M)$ depends on $\text{head}(M)$. If $\text{head}(M) = b$, then throw it away and return the value of $\text{removeAll}(b, \text{tail}(M))$. But if $\text{head}(M) \neq b$, then it’s a keeper. So we should return the value $\text{head}(M) :: \text{removeAll}(b, \text{tail}(M))$. We can write the definition in if-then-else form as follows:

$$\begin{aligned}\text{removeAll}(b, M) &= \text{if } M = \langle \rangle \text{ then } \langle \rangle \\ &\quad \text{else if head}(M) = b \text{ then} \\ &\quad \quad \text{removeAll}(b, \text{tail}(M)) \\ &\quad \text{else} \\ &\quad \quad \text{head}(M) :: \text{removeAll}(b, \text{tail}(M)).\end{aligned}$$

We’ll evaluate the expression $\text{removeAll}(b, a, b, c, b)$:

$$\begin{aligned}
\text{removeAll}(b, \langle a, b, c, b \rangle) &= a :: \text{removeAll}(b, \langle b, c, b \rangle) \\
&= a :: \text{removeAll}(b, \langle c, b \rangle) \\
&= a :: c :: \text{removeAll}(b, \langle b \rangle) \\
&= a :: c :: \text{removeAll}(b, \langle \rangle) \\
&= a :: c :: \langle \rangle \\
&= a :: \langle c \rangle \\
&= \langle a, c \rangle.
\end{aligned}$$

Try to write out each unfolding step in the evaluation of the expression $\text{remove}(\langle b, a, b \rangle)$. Be sure to start writing at the left-hand edge of your paper.

The Power Set Problem

Suppose we want to construct the power set of a finite set. One solution uses the fact that $\text{power}(\{x\} \cup T)$ is the union of $\text{power}(T)$ and the set obtained from $\text{power}(T)$ by adding x to each of its elements. Let's see whether we can discover a solution technique by considering a small example. Let $S = \{a, b, c\}$. Then we can write $\text{power}(S)$ as follows:

$$\begin{aligned}
\text{power}(S) &= \{\{\}, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\} \\
&= \{\{\}, \{b\}, \{c\}, \{b, c\}\} \cup \{\{a\}, \{a, b\}, \{a, c\}, \{a, b, c\}\}.
\end{aligned}$$

We've written $\text{power}(S) = A \cup B$, where B is obtained from A by adding the element a to each set in A . If we represent S as the list $\langle a, b, c \rangle$, then we can restate the definition for $\text{power}(S)$ as the concatenation of the following two lists:

$$\langle \langle \rangle, \langle b \rangle, \langle c \rangle, \langle b, c \rangle \rangle \quad \text{and} \quad \langle \langle a \rangle, \langle a, b \rangle, \langle a, c \rangle, \langle a, b, c \rangle \rangle.$$

The first of these lists is $\text{power}(\langle b, c \rangle)$. The second list can be obtained from $\text{power}(\langle b, c \rangle)$ by working backward to the answer as follows:

$$\begin{aligned}
\langle \langle a \rangle, \langle a, b \rangle, \langle a, c \rangle, \langle a, b, c \rangle \rangle &= \langle a :: \langle \rangle, a :: \langle b \rangle, a :: \langle c \rangle, a :: \langle b, c \rangle \rangle \\
&= \text{map} (::, \langle \langle a, \langle \rangle \rangle, \langle a, \langle b \rangle \rangle, \langle a, \langle c \rangle \rangle, \langle a, \langle b, c \rangle \rangle \rangle) \\
&= \text{map} (::, \text{dist} (a, \text{power} (\langle b, c \rangle))) .
\end{aligned}$$

This example is the key to the recursive part of the definition. Using the fact that $\text{power}(\langle \rangle) = \langle \langle \rangle \rangle$ as the basis case, we can write down the following definition for power:

$$\begin{aligned}
\text{power} (\langle \rangle) &= \langle \langle \rangle \rangle, \\
\text{power} (a :: T) &= \text{cat} (\text{power} (T), \text{map} (::, \text{dist} (a, \text{power} (T)))) .
\end{aligned}$$

The if-then-else form of the definition can be written as follows:

$$\begin{aligned}
\text{power} (L) &= \text{if } L = \langle \rangle \text{ then } \langle \langle \rangle \rangle \text{ else} \\
&\quad \text{cat} (\text{power} (\text{tail} (L)), \text{map} (::, \text{dist} (\text{head} (L), \text{power} (\text{tail} (L))))) .
\end{aligned}$$

We'll evaluate the expression $\text{power} (a, b)$. The first step yields the equation

$$\text{power} (a, b) = \text{cat} (\text{power} (b), \text{map} (::, \text{dist} (a, \text{power} (b)))) .$$

Now we'll evaluate $\text{power} (b)$ and substitute it in the preceding equation:

$$\begin{aligned}
\text{power} (\langle b \rangle) &= \text{cat} (\text{power} (\langle \rangle), \text{map} (::, \text{dist} (b, \text{power} (\langle \rangle)))) \\
&= \text{cat} (\langle \langle \rangle \rangle, \text{map} (::, \text{dist} (b, \langle \langle \rangle \rangle))) \\
&= \text{cat} (\langle \langle \rangle \rangle, \text{map} (::, \langle \langle b, \langle \rangle \rangle \rangle)) \\
&= \text{cat} (\langle \langle \rangle \rangle, \langle b :: \langle \rangle \rangle) \\
&= \text{cat} (\langle \langle \rangle \rangle, \langle \langle b \rangle \rangle) \\
&= \langle \langle \rangle, \langle b \rangle \rangle .
\end{aligned}$$

Now we can continue with the evaluation of $\text{power} (a, b)$:

$$\begin{aligned}
\text{power}(\langle a, b \rangle) &= \text{cat}(\text{power}(\langle b \rangle), \text{map}(\text{::}, \text{dist}(a, \text{power}(\langle b \rangle)))) \\
&= \text{cat}(\langle \langle \rangle, \langle b \rangle \rangle, \text{map}(\text{::}, \text{dist}(a, \langle \langle \rangle, \langle b \rangle \rangle))) \\
&= \text{cat}(\langle \langle \rangle, \langle b \rangle \rangle, \text{map}(\text{::}, \langle \langle a, \langle \rangle \rangle, \langle a, \langle b \rangle \rangle \rangle)) \\
&= \text{cat}(\langle \langle \rangle, \langle b \rangle \rangle, \langle a \text{ :: } \langle \rangle, a \text{ :: } \langle b \rangle \rangle) \\
&= \text{cat}(\langle \langle \rangle, \langle b \rangle \rangle, \langle \langle a \rangle, \langle a, b \rangle \rangle) \\
&= \langle \langle \rangle, \langle b \rangle, \langle a \rangle, \langle a, b \rangle \rangle.
\end{aligned}$$

3.2.6 Infinite Sequences

Let's see how some infinite sequences can be defined recursively. To illustrate the idea, suppose the function “ints” returns the following infinite sequence for any integer x :

$$\text{ints}(x) = \langle x, x + 1, x + 2, \dots \rangle.$$

We'll assume that the list operations of cons, head, and tail work for infinite sequences. For example, the following relationships hold.

$$\begin{aligned}
\text{ints}(x) &= x \text{ :: ints}(x + 1), \\
\text{head}(\text{ints}(x)) &= x, \\
\text{tail}(\text{ints}(x)) &= \text{ints}(x + 1).
\end{aligned}$$

Even though the definition of ints does not conform to (3.6), it is still recursively defined because it is defined in terms of itself. If we executed the definition, an infinite loop would construct the infinite sequence. For example, $\text{ints}(0)$ would construct the infinite sequence of natural numbers as follows:

$$\begin{aligned}
\text{ints}(0) &= 0 \text{ :: ints}(1) \\
&= 0 \text{ :: } 1 \text{ :: ints}(2) \\
&= 0 \text{ :: } 1 \text{ :: } 2 \text{ :: ints}(3) \\
&= \dots
\end{aligned}$$

In practice, an infinite sequence is used as an argument and it is evaluated only

when some of its values are needed. Once the needed values are computed, the evaluation stops. This is an example of a technique called *lazy evaluation*. For example, the following function returns the n th element of an infinite sequence s .

$$\text{get}(n, s) = \text{if } n = 1 \text{ then head}(s) \text{ else get}(n - 1, \text{tail}(s)).$$

example 3.31 Picking Elements

We'll get the third element from the infinite sequence `ints(6)` by unfolding the expression `get(3, ints(6))`.

$$\begin{aligned} \text{get}(3, \text{ints}(6)) &= \text{get}(2, \text{tail}(\text{ints}(6))) \\ &= \text{get}(1, \text{tail}(\text{tail}(\text{ints}(6)))) \\ &= \text{head}(\text{tail}(\text{tail}(\text{ints}(6)))) \\ &= \text{head}(\text{tail}(\text{tail}(6 :: \text{ints}(7)))) \\ &= \text{head}(\text{tail}(\text{ints}(7))) \\ &= \text{head}(\text{tail}(7 :: \text{ints}(8))) \\ &= \text{head}(\text{ints}(8)) \\ &= \text{head}(8 :: \text{ints}(9)) \\ &= 8. \end{aligned}$$

end example

example 3.32 Summing

Suppose we need a function to sum the first n elements in an infinite sequence s of integers. The following definition does the trick:

$$\text{sum}(n, s) = \text{if } n = 0 \text{ then } 0 \text{ else head}(s) + \text{sum}(n - 1, \text{tail}(s)).$$

We'll compute the sum of the first three numbers in `ints(4)`:

$$\begin{aligned}
\text{sum}(3, \text{ints}(4)) &= 4 + \text{sum}(2, \text{ints}(5)) \\
&= 4 + 5 + \text{sum}(1, \text{ints}(6)) \\
&= 4 + 5 + 6 + \text{sum}(0, \text{ints}(7)) \\
&= 4 + 5 + 6 + 0 \\
&= 15.
\end{aligned}$$

end example

example 3.33 The Sieve of Eratosthenes

Suppose we want to study prime numbers. For example, we might want to find the 500th prime, we might want to find the difference between the 500th and 501st primes, and so on. One way to proceed might be to define functions to extract information from the following infinite sequence of all prime numbers.

$$\text{Primes} = \langle 2, 3, 5, 7, 11, 13, 17, \dots \rangle.$$

We'll construct this infinite sequence by the method of Eratosthenes (called *the sieve of Eratosthenes*). The method starts with the infinite sequence $\text{ints}(2)$:

$$\text{ints}(2) = \langle 2, 3, 4, 5, 6, 7, 8, 9, 10, \dots \rangle.$$

The next step removes all multiples of 2 (except 2) to obtain the infinite sequence

$$\langle 2, 3, 5, 7, 9, 11, 13, 15, \dots \rangle.$$

The next step removes all multiples of 3 (except 3) to obtain the infinite sequence

$$\langle 2, 3, 5, 7, 11, 13, 17, \dots \rangle.$$

The process continues in this way.

We can construct the desired infinite sequence of primes once we have the function to remove multiples of a number from an infinite sequence. If we let $\text{remove}(n, s)$ denote the infinite sequence obtained from s by removing all

multiples of n , then we can define the sieve process as follows for an infinite sequence s of numbers:

$$\text{sieve}(s) = \text{head}(s) :: \text{sieve}(\text{remove}(\text{head}(s), \text{tail}(s))).$$

But we need to define the remove function. Notice that for natural numbers m and n with $n > 0$ that we have the following equivalences:

$$m \text{ is a multiple of } n \text{ iff } n \text{ divides } m \text{ iff } m \bmod n = 0.$$

This allows us to write the following definition for the remove function:

$$\begin{aligned} \text{remove}(n, s) = & \text{if } \text{head}(s) \bmod n = 0 \text{ then } \text{remove}(n, \text{tail}(s)) \\ & \text{else } \text{head}(s) :: \text{remove}(n, \text{tail}(s)). \end{aligned}$$

Then our desired sequence of primes is represented by the expression

$$\text{Primes} = \text{sieve}(\text{ints}(2)).$$

In the exercises we'll evaluate some functions dealing with primes.

end example

Exercises

Evaluating Recursively Defined Functions

1. Given the following definition for the n th Fibonacci number:

$$\begin{aligned} \text{fib}(0) &= 0, \\ \text{fib}(1) &= 1, \\ \text{fib}(n) &= \text{fib}(n-1) + \text{fib}(n-2) \quad \text{if } n > 1. \end{aligned}$$

Write down each step in the evaluation of $\text{fib}(4)$.

2. Given the following definition for the length of a list:

$$\text{length}(L) = \text{if } L = \langle \rangle \text{ then } 0 \text{ else } 1 + \text{length}(\text{tail}(L)).$$

Write down each step in the evaluation of $\text{length}(\langle r, s, t, u \rangle)$.

3. For each of the two definitions of “makeTree” given by (3.9) and (3.10), write down all steps to evaluate $\text{makeTree}(\quad, 3, 2, 4)$.

Numbers

4. Construct a recursive definition for each of the following functions, where all variables represent natural numbers.

- a. $f(n) = 0 + 2 + 4 + \cdots + 2n$.
- b. $f(n) = \text{floor}(0/2) + \text{floor}(1/2) + \cdots + \text{floor}(n/2)$.
- c. $f(k, n) = \text{gcd}(1, n) + \text{gcd}(2, n) + \cdots + \text{gcd}(k, n)$ for $k > 0$.
- d. $f(n) = (0 \bmod 2) + (1 \bmod 3) + \cdots + (n \bmod (n + 2))$.
- e. $f(n, k) = 0 + k + 2k + \cdots + nk$.
- f. $f(n, k) = k + (k + 1) + (k + 2) + \cdots + (k + n)$.

Strings

5. Construct a recursive definition for each of the following string functions for strings over the alphabet $\{a, b\}$.

- a. $f(x)$ returns the reverse of x .
- b. $f(x) = xy$, where y is the reverse of x .
- c. $f(x, y)$ tests whether x is a prefix of y .
- d. $f(x, y)$ tests whether $x = y$.
- e. $f(x)$ tests whether x is a palindrome.

Lists

6. Construct a recursive definition for each of the following functions that involve lists. Use the infix form of cons in the recursive part of each definition. In other words, write $h :: t$ in place of $\text{cons}(h, t)$.

- a. $f(n) = 2n, 2(n - 1), \dots, 2, 0$, over natural numbers.
- b. $f(z) = \text{if } z \geq 0 \text{ then } z, z - 1, \dots, 0 \text{ else } z, z + 1, \dots, 0$, over integers.

- c. $\text{max}(L)$ is the maximum value in the nonempty list L of numbers.
- d. $f(x, \langle a_0, \dots, a_n \rangle) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$.
- e. $f(L)$ is the list of elements x in list L that have property P .
- f. $f(a, \langle x_1, \dots, x_n \rangle) = \langle x_1 + a, \dots, x_n + a \rangle$.
- g. $f(a, \langle (x_1, y_1), \dots, (x_n, y_n) \rangle) = \langle (x_1 + a, y_1), \dots, (x_n + a, y_n) \rangle$.
- h. $f(g, \langle x_1, \dots, x_n \rangle) = \langle (x_1, g(x_1)), \dots, (x_n, g(x_n)) \rangle$.
- i. $f(n) = \langle (0, n), (1, n-1), \dots, (n-1, 1), (n, 0) \rangle$. *Hint: Use Part (g).*
- j. $f(g, h, \langle x_1, \dots, x_n \rangle) = \langle (g(x_1), h(x_1)), \dots, (g(x_n), h(x_n)) \rangle$.

Using Cat or ConsR

7. Construct a recursive definition for each of the following functions that involve lists. Use the cat operation or consR operation in the recursive part of each definition. (Notice that for any list L and element x we have $\text{cat}(L, \langle x \rangle) = \text{consR}(L, x)$.)

- a. $f(n) = \langle 0, 1, \dots, n \rangle$.
- b. $f(n) = \langle 0, 2, 4, \dots, 2n \rangle$.
- c. $f(n) = \langle 1, 3, 5, \dots, 2n+1 \rangle$.
- d. $f(n, k) = \langle n, n+1, n+2, \dots, n+k \rangle$.
- e. $f(n, k) = \langle 0, k, 2k, 3k, \dots, nk \rangle$.
- f. $f(g, n) = \langle (0, g(0)), (1, g(1)), \dots, (n, g(n)) \rangle$.
- g. $f(n, m) = \langle n, n+1, n+2, \dots, m-1, m \rangle$, where $n \leq m$.

8. Let *insert* be a function that extends any binary function so that it evaluates a list of two or more arguments. For example,

$$\text{insert}(+, \langle 1, 4, 2, 9 \rangle) = 1 + (4 + (2 + 9)) = 16.$$

Write a recursive definition for $\text{insert}(f, L)$, where f is any binary function and L is a list of two or more arguments.

9. Write a recursive definition for the function “eq” to check two lists for equality.

10. Write recursive definitions for the following list functions.
 - a. The function “last” that returns the last element of a nonempty list. For example, $\text{last}(\langle a, b, c \rangle) = c$.
 - b. The function “front” that returns the list obtained by removing the last element of a nonempty list. For example, $\text{front}(\langle a, b, c \rangle) = \langle a, b \rangle$.
11. Write down a recursive definition for the function “pal” that tests a list of letters to see whether their concatenations form a palindrome. For example, $\text{pal}(\langle r, a, d, a, r \rangle) = \text{true}$ since *radar* is a palindrome. *Hint*: Use the functions of Exercise 10.
12. Solve the repeated element problem with the restriction that we want to keep the rightmost occurrence of each repeated element. *Hint*: Use the functions of Exercise 10.

Binary Trees

13. Given the algebraic expression $a + (b \cdot (d + e))$, draw a picture of the binary tree representation of the expression. Then write down the preorder, inorder, and postorder listings of the tree. Are any of the listings familiar to you?
14. Write down recursive definitions for each of the following procedures to print the nodes of a binary tree.
 - a. In: Prints the nodes of a binary tree from an inorder traversal.
 - b. Post: Prints the nodes of a binary tree from a postorder traversal.
15. Write down recursive definitions for each of the following functions. Include both the pattern-matching and if-then-else forms for each definition.
 - a. leaves: Returns the number of leaf nodes in a binary tree.
 - b. inOrd: Returns the inorder listing of nodes in a binary tree.
 - c. postOrd: Returns the postorder listing of nodes in a binary tree.
16. Construct a recursive definition for each of the following functions that involve binary trees. Represent binary trees as lists where $\langle \rangle$ is the empty tree and any nonempty binary tree has the form $\langle L, r, R \rangle$, where r is the root

and L and R are its left and right subtrees. Assume that nodes are numbers.

- a. $f(T)$ is the sum of the nodes of T .
- b. $f(T)$ is the depth of T , where the empty tree has depth -1 .
- c. $f(T)$ is the list of nodes in T that have property p .
- d. $f(T)$ is the maximum value of the nodes in the nonempty binary tree T .

Trees and Algebraic Expressions

17. Recall from Section 1.4 that any algebraic expression can be represented as a tree and the tree can be represented as a list whose head is the root and whose tail is the list of operands in the form of trees. For example, the algebraic expression $a \times b + f(c, d, e)$, can be represented by the list

$$\langle +, \times, a, b, f, c, d, e \rangle.$$

- a. Draw the picture of the tree for the given algebraic expression.
- b. Construct a recursive definition for the function “post” that takes an algebraic expression written in the form of a list, as above, and returns the list of nodes in the algebraic expression tree in postfix notation. For example,

$$\text{post}(\langle +, \times, a, b, f, c, d, e \rangle) = \langle a, b, \times, c, d, e, f, + \rangle.$$

Relations as Lists of Tuples

18. Construct a recursive definition for each of the following functions that involve lists of tuples. If x is an n -tuple, then x_k represents the k th component of x .
- a. $f(k, L)$ is the list of k th components x_k of tuples x in the list L .
 - b. $\text{sel}(k, a, L)$ is the list of tuples x in the list L such that $x_k = a$.

Sets Represented as Lists

19. Write a recursive definition for each of the following functions, in which the input arguments are sets represented as lists. Use the primitive operations of cons, head, and tail to build your functions (along with

functions already defined):

- a. `isMember`. For example, `isMember(a, <b, a, c>)` is true.
- b. `isSubset`. For example, `isSubset(<a, b>, <b, c, a>)` is true.
- c. `areEqual`. For example, `areEqual(<a, b>, <b, a>)` is true.
- d. `union`. For example, `union(<a, b>, <c, a>) = <a, b, c>`.
- e. `intersect`. For example, `intersect(<a, b>, <c, a>) = <a>`.
- f. `difference`. For example, `difference(<a, b, c>, <b, d>) = <a, c>`.

Challenges

20. Conway's challenge sequence is defined recursively as follows:

Basis: $f(1) = f(2) = 1$.

Recursion: $f(n) = f(f(n-1)) + f(n - f(n-1))$ for $n > 2$.

Calculate the first 17 elements $f(1), f(2), \dots, f(17)$. The article by Mallows [1991] contains an account of this sequence.

21. Let $\text{fib}(k)$ denote the k th Fibonacci number, and let

$$\text{sum}(k) = 1 + 2 + \dots + k.$$

Write a recursive definition for the function $f: \mathbb{N} \rightarrow \mathbb{N}$ defined by $f(n) = \text{sum}(\text{fib}(n))$. *Hint:* Write down several examples, such as $f(0), f(1), f(2), f(3), f(4), \dots$. Then try to find a way to write $f(4)$ in terms of $f(3)$. This might help you discover a pattern.

22. Write a function in if-then-else form to produce the Cartesian product set of two finite sets. You may assume that the sets are represented as lists.

23. We can approximate the square root of a number by using the Newton–Raphson method, which gives an infinite sequence of approximations to the square root of x by starting with an initial guess g . We can define the sequence with the following function:

$$\text{sqrt}(x, g) = g :: \text{sqrt}(x, (0.5)(g + (x/g))).$$

Find the first three numbers in each of the following infinite sequences,

and compare the values with the square root obtained with a calculator.

- a. $\text{sqrt}(4, 1)$.
- b. $\text{sqrt}(4, 2)$.
- c. $\text{sqrt}(4, 3)$.
- d. $\text{sqrt}(2, 1)$.
- e. $\text{sqrt}(9, 1)$.
- f. $\text{sqrt}(9, 5)$.

24. Find a definition for each of the following infinite sequence functions.

- a. Square: Squares each element in a sequence of numbers.
- b. Diff: Finds the difference of the n th and m th numbers of a sequence.
- c. Prod: Finds the product of the first n numbers of a sequence.
- d. Add: Adds corresponding elements of two numeric sequences.
- e. $\text{Skip}(x, k) = \langle x, x + k, x + 2k, x + 3k, \dots \rangle$.
- f. Map: Applies a function to each element of a sequence.
- g. ListOf: Finds the list of the first n elements of a sequence.

25. Evaluate each of the following expressions by unfolding the definitions for *Primes* and remove from Example 3.33.

- a. $\text{head}(\text{Primes})$.
- b. $\text{tail}(\text{Primes})$ until reaching the value $\text{sieve}(\text{remove}(2, \text{ints}(3)))$.
- c. $\text{remove}(2, \text{ints}(0))$ until reaching the value $1 :: 2 :: \text{remove}(2, \text{ints}(4))$.

26. Suppose we define the function $f: \mathbb{N} \rightarrow \mathbb{N}$ by

$$f(x) = \text{if } x > 10 \text{ then } x - 10 \text{ else } f(f(x + 11)).$$

This function is recursively defined even though it is not defined by (3.6). Give a simple definition of the function.

3.3 Grammars

Informally, a grammar is a set of rules used to define the structure of the strings